



清华大学计算机科学与技术系本科专业主修课

计算机系统结构

第6章 指令级并行及其开发—软件方法

2026年春季学期

上一讲回顾

第 2 页

- **指令级并行开发（硬件方法）**：依靠专门硬件对代码进行调度，以减少数据相关导致的性能下降。

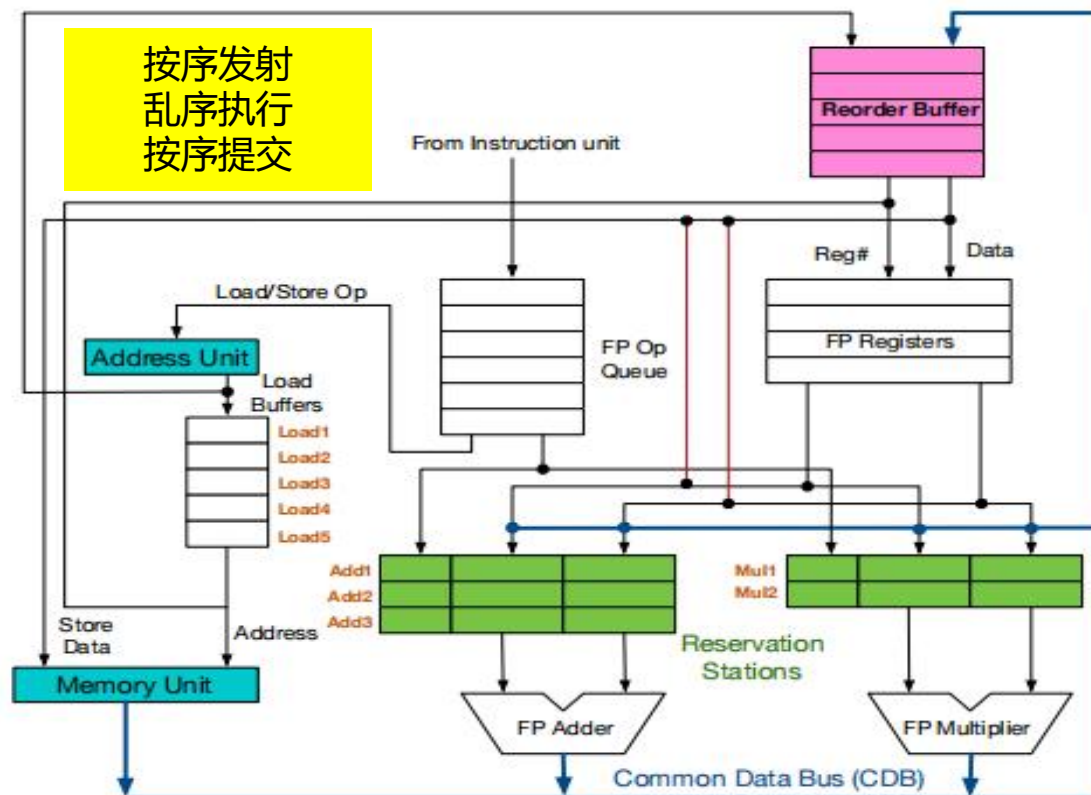
记分牌算法+显式重命名

指令		指令状态表			
		流出	读操作数	执行	写结果
L.D	F6, 34(R2)	1	按序发射 (issue) 乱序执行 (execute) 和提交(commit)		4
L.D	F2, 45(R3)	5			8
MULT.D	F0, F2, F4	6			20
SUB.D	F8, F6, F2	7			12
DIV.D	F10, F0, F6	8	21	61	62
ADD.D	F6, F8, F2	13	14	16	21

部件名称	功能部件状态表								
	Busy	Op	F _i	F _j	F _k	Q _j	Q _k	R _j	R _k
Integer	No								
Mult1	No								
Mult2	No								
Add	No								
Divide	No								

部件名称	结果寄存器状态表							
	F0	F2	F4	F6	F8	F10	...	F30
部件名称								

Tomasulo算法+重排序缓存ROB



■ 硬件方法的基础：指令级并行设计对象

- **基本程序块**：一串连续的代码（同时执行相邻或相近的多条指令），除了入口和出口以外没有其他的分支指令和转入点

- **分支指令和转入点**：程序平均每4 ~ 7条指令就会有一个分支

- 循环级并行是指令级并行研究的重点之一

■ 存在问题：如何开发更多的指令级并行？

■ 核心方法：突破基本程序块层面的指令调度

- 循环展开

- 跨跃基本程序块的指令调度等

} **软件方法**

- 6. 1 [基本指令调度及循环展开](#)
- 6. 2 [跨越基本块的静态指令调度](#)
- 6. 3 [静态多指令流出：VLIW技术](#)
- 6. 4 [显式并行指令计算EPIC](#)
- 6. 5 [开发更多的指令级并行](#)

6. 1 基本指令调度及循环展开

6.1.1 指令调度的基本方法

第 6 页

1. 指令调度：找出不相关的指令序列，让它们在流水线上重叠并行执行。
2. 制约编译器指令调度的因素
 - 程序固有的指令级并行
 - 流水线功能部件的延迟

表6.1 本节使用的浮点流水线的延迟

产生结果的指令	使用结果的指令	延迟(cycles)
浮点计算	另一个浮点计算	3
浮点计算	浮点store(S.D)	2
浮点Load(L.D)	浮点计算	1
浮点Load(L.D)	浮点store(S.D)	0

6.1.1 指令调度的基本方法

第 7 页

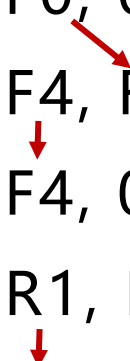
例6.1 对于下面的源代码，转换成MIPS汇编语言，在不进行指令调度和进行指令调度两种情况下，分析其代码一次循环所需的执行时间。

```
for (i=1000; i>0; i--)
```

```
    x[i] = x[i] + s;
```

解：先把该程序翻译成MIPS汇编语言代码

```
Loop: L.D      F0, 0(R1)
      ADD.D    F4, F0, F2
      S.D      F4, 0(R1)
      DADDIU   R1, R1, #-8
      BNE     R1, R2, Loop
```



数据相关：
L.D与ADD.D
ADD.D与S.D
DADDIU与BNE

6.1.1 指令调度的基本方法

第 8 页

在不进行指令调度的情况下，根据表中给出的浮点流水线中指令执行的延迟，程序的实际执行情况如下：

		指令流出时钟
Loop: L.D	F0, 0(R1)	1
(空转)		2
ADD.D	F4, F0, F2	3
(空转)		4
(空转)		5
S.D	F4, 0(R1)	6
DADDIU	R1, R1, #-8	7
(空转)		8
BNE	R1, R2, Loop	9
(空转)		10

产生结果的指令	使用结果的指令	延迟(cycles)
浮点计算	另一个浮点计算	3
浮点计算	浮点store(S.D)	2
浮点Load(L.D)	浮点计算	1
浮点Load(L.D)	浮点store(S.D)	0

完成一个循环体共10个周期，其中5个空转周期。如果完成4个循环，共需要40个周期

6.1.1 指令调度的基本方法

第 9 页

在用编译器对上述程序进行指令调度以后，程序的执行情况如下：

指令流出时钟

Loop:	L.D	F0, 0(R1)	1	完成一个循环体共6个周期， 其中1个空转周期
	DADDIU	R1, R1, #-8	2	
	ADD.D	F4, F0, F2	3	
	(空转)		4	
	BNE	R1, R2, Loop	5	
	S.D	F4, 8(R1)	6	

注意：（1）把S.D指令放到了分支指令的延迟槽中；

（2）由于修改指令的DADDIU指令（被调度到S.D指令之前），提前对指令进行了减8操作，所以要对S.D指令中的偏移量进行修正，即把“0(R1)”改成“**8(R1)**”；

（3）编译时指令调度不能消除指令间的相关，而是通过重新安排指令的流出顺序（如S.D、DADDIU指令）尽可能少地引起流水线空转，进而减少整个指令序列的执行时间。

6.1.1 指令调度的基本方法

第 10 页

Loop: L.D F0, 0(R1)
 DADDIU R1, R1, #-8
 ADD.D F4, F0, F2
 (空转)
 BNE R1, R2, Loop
 S.D F4, 8(R1)

问题1：在基本指令调度方法中，指令调度不能跨越分支指令。这里需要说明的是，S.D指令被调度到BNE指令的分支延迟槽中不属于跨跃分支指令的情况；

问题2：指令序列中，只有L.D、S.D、ADD.D是有效操作，占3个周期，其他指令为控制循环与解决数据相关等待而附加的，整个指令序列的有效操作比例不高，**3/6=50%**。

目标：如何进一步减少整个指令序列的执行时间：

- 方法一：指令调度能否跨越分支边界？
- 方法二：怎样提高整个执行过程中有效操作的比率？

循环展开

1. 循环展开

- 把循环体的代码复制多次并按顺序排放，然后相应调整循环的结束条件。
- 开发循环级并行的有效方法

例6.2 将例6.1中的**循环展开3次得到4个循环体**，然后对展开后的指令序列在不调度和调度两种情况下，分析代码的性能。

假定R1的初值为32的倍数，即循环次数为4的倍数。要求：消除冗余的指令，并且不要重复使用寄存器。

6.1.2 循环展开

展开后没有调度的代码如下（循环展开+重分配寄存器，无指令调度）

	指令流出时钟		指令流出时钟
Loop: L.D F0, 0(R1)	1	ADD.D F12, F10, F2	15
(空转)	2	(空转)	16
ADD.D F4, F0, F2	3	(空转)	17
(空转)	4	S.D F12, -16 (R1)	18
(空转)	5	L.D F14, -24 (R1)	19
S.D F4, 0(R1)	6	(空转)	20
L.D F6, -8(R1)	7	ADD.D F16, F14, F2	21
(空转)	8	(空转)	22
ADD.D F8, F6, F2	9	(空转)	23
(空转)	10	S.D F16, -24 (R1)	24
(空转)	11	DADDIU R1, R1, # -32	25
S.D F8, -8(R1)	12	(空转)	26
L.D F10, -16(R1)	13	BNE R1, R2, Loop	27
(空转)	14	(空转)	28

注意：
(1) 重新分配寄存器 F0、F4、F6、F8等，不是寄存器重命名；
(2) DADDIU、L.D和 S.D的指令的偏移量进行相应的修正；
(3) 这个循环共执行 28个时钟周期，平均每个循环体用 28/4=7<10个时钟周期；
(4) 实际指令14条，空转14个时钟周期，执行效率不高。

6.1.2 循环展开

调度后的代码如下:

		指令流出时钟
Loop:	L.D	F0, 0 (R1) 1
	L.D	F6, -8 (R1) 2
	L.D	F10, -16 (R1) 3
	L.D	F14, -24 (R1) 4
	ADD.D	F4, F0, F2 5
	ADD.D	F8, F6, F2 6
	ADD.D	F12, F10, F2 7
	ADD.D	F16, F14, F2 8
	S.D	F4, 0 (R1) 9
	S.D	F8, -8 (R1) 10
	DADDIU	R1, R1, # -32 12
	S.D	F12, 16 (R1) 11
	BNE	R1, R2, Loop 13
	S.D	F16, 8 (R1) 14

4个循环共执行14个时钟周期，平均每个循环体用14/4=3.5个时钟周期

没有空转周期!

4个循环的执行时间:
40->28->14

通过循环展开、寄存器重分配和指令调度，增大基本程序块，可以更有效开发出指令级并行。

6.1.2 循环展开

第 14 页

■ 循环展开和指令调度的注意事项

- 保证正确性：循环控制 and 操作数偏移量的修改

指令流出时钟			指令流出时钟			
Loop:	L.D	F0, 0(R1)	1	ADD.D	F12, F10, F2	15
	(空转)		2	(空转)		16
	ADD.D	F4, F0, F2	3	(空转)		17
	(空转)		4	S.D	F12, -16 (R1)	18
	(空转)		5	L.D	F14, -24 (R1)	19
	S.D	F4, 0(R1)	6	(空转)		20
	L.D	F6, -8(R1)	7	ADD.D	F16, F14, F2	21
	(空转)		8	(空转)		22
	ADD.D	F8, F6, F2	9	(空转)		23
	(空转)		10	S.D	F16, -24 (R1)	24
	(空转)		11	DADDIUR1, R1, # -32	25	
	S.D	F8, -8(R1)	12	(空转)		26
	L.D	F10, -16(R1)	13	BNE	R1, R2, Loop	27
	(空转)		14	(空转)		28

6.1.2 循环展开

■ 循环展开和指令调度的注意事项

➤ 注意有效性：只有找到不同循环体之间的无关性，才能有效使用循环展开

```
for (i=1000; i>0; i--)  
    x[i+1] = x[i] + s;  
  
Loop:  L.D      F0, 0(R1)  
        ADD.D   F4, F0, F2  
        S.D     F4, -8(R1)  
        DADDIU  R1, R1, #-8  
        BNE     R1, R2, Loop
```



Loop:	L.D	F0, 0(R1)	1	指令流出时钟
	(空转)		2	
	ADD.D	F4, F0, F2	3	
	(空转)		4	
	(空转)		5	
	S.D	F4, -8(R1)	6	
	L.D	F6, -8(R1)	7	
	(空转)		8	
	ADD.D	F8, F6, F2	9	
	(空转)		10	
	(空转)		11	
	S.D	F8, -16(R1)	12	
	L.D	F10, -16(R1)	13	
	(空转)		14	

	ADD.D	F12, F10, F2	15	指令流出时钟
	(空转)		16	
	(空转)		17	
	S.D	F12, -24 (R1)	18	
	L.D	F14, -24 (R1)	19	
	(空转)		20	
	ADD.D	F16, F14, F2	21	
	(空转)		22	
	(空转)		23	
	S.D	F16, -32 (R1)	24	
	DADDIU	R1, R1, #-32	25	
	(空转)		26	
	BNE	R1, R2, Loop	27	
	(空转)		28	



6.1.2 循环展开

第 16 页

■ 循环展开和指令调度的注意事项

- 使用不同的寄存器：重新分配寄存器，以避免产生新的冲突

指令流出时钟			指令流出时钟			
Loop:	L.D	F0, 0(R1)	1	ADD.D	F12, F10, F2	15
	(空转)		2	(空转)		16
	ADD.D	F4, F0, F2	3	(空转)		17
	(空转)		4	S.D	F12, -16 (R1)	18
	(空转)		5	L.D	F14, -24 (R1)	19
	S.D	F4, 0(R1)	6	(空转)		20
	L.D	F6, -8(R1)	7	ADD.D	F16, F14, F2	21
	(空转)		8	(空转)		22
	ADD.D	F8, F6, F2	9	(空转)		23
	(空转)		10	S.D	F16, -24 (R1)	24
	(空转)		11	DADDIUR1, R1, # -32	25	
	S.D	F8, -8(R1)	12	(空转)		26
	L.D	F10, -16(R1)	13	BNE	R1, R2, Loop	27
	(空转)		14	(空转)		28

6.1.2 循环展开

第 17 页

■ 循环展开和指令调度的注意事项

- 删除多余的测试指令和分支指令，并对循环结束代码和新的循环体代码进行相应的修正

指令流出时钟			指令流出时钟			
Loop:	L.D	F0, 0(R1)	1	ADD.D	F12, F10, F2	15
	(空转)		2	(空转)		16
	ADD.D	F4, F0, F2	3	(空转)		17
	(空转)		4	S.D	F12, -16 (R1)	18
	(空转)		5	L.D	F14, -24 (R1)	19
	S.D	F4, 0(R1)	6	(空转)		20
	L.D	F6, -8(R1)	7	ADD.D	F16, F14, F2	21
	(空转)		8	(空转)		22
	ADD.D	F8, F6, F2	9	(空转)		23
	(空转)		10	S.D	F16, -24 (R1)	24
	(空转)		11	DADDIUR1, R1, # -32	25	
	S.D	F8, -8(R1)	12	(空转)		26
	L.D	F10, -16(R1)	13	BNE	R1, R2, Loop	27
	(空转)		14	(空转)		28

6.1.2 循环展开

第 18 页

■ 循环展开和指令调度的注意事项

- 注意对存储器数据的相关性分析，例如不同循环体中的存储器地址不同，则可以相互对调
- 注意新的相关性，如新的WAW、WAR等

指令流出时钟

Loop:	L.D	F0, 0 (R1)	1
	L.D	F6, -8 (R1)	2
	L.D	F10, -16 (R1)	3
	L.D	F14, -24 (R1)	4
	ADD.D	F4, F0, F2	5
	ADD.D	F8, F6, F2	6
	ADD.D	F12, F10, F2	7
	ADD.D	F16, F14, F2	8
	S.D	F4, 0 (R1)	9
	S.D	F8, -8 (R1)	10
	DADDIU	R1, R1, # -32	12
	S.D	F12, 16 (R1)	11
	BNE	R1, R2, Loop	13
	S.D	F16, 8 (R1)	14

6.2 跨越基本块的静态指令调度

6.2.1 全局指令调度

第 20 页

■ 循环展开存在问题

Loop:	L.D	F0, 0(R1)	}	循环体是顺序结构且结构相同
	ADD.D	F4, F0, F2		
	S.D	F4, 0(R1)		
	DADDIU	R1, R1, #-8		
	BNE	R1, R2, Loop		

```
if (条件) {  
    // 条件为真时执行的代码  
}  
else {  
    // 条件为假时执行的代码  
}
```

如果程序是分支结构，则优化含有分支结构的程序需要在多个（不同）基本块间移动指令，这种调度技术被称为“全局指令调度”，主要包括：

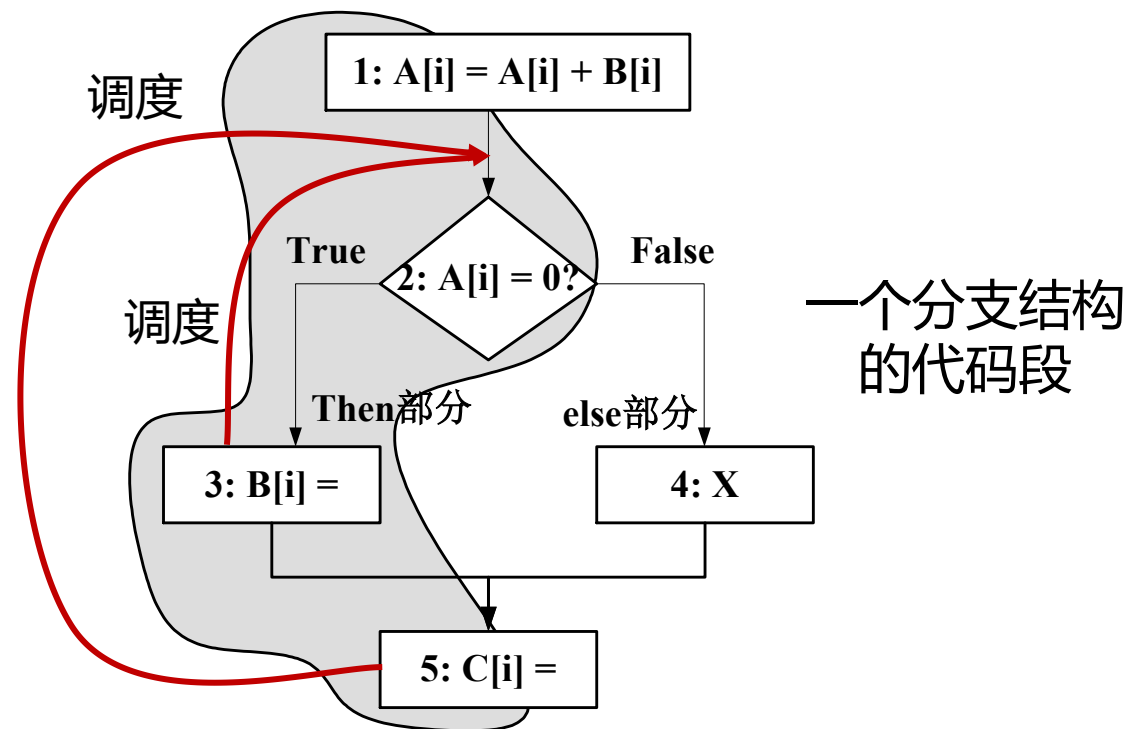
- 踪迹调度
- 超块调度

6.2.1 全局指令调度

第 21 页

1. 实例分析

- 分析指令转移成功和失败的概率一般不同
- 由于分支条件为true(转移)的概率大，全局指令调度时会把语句1、2、3、5合并为一个更大的基本块。



基本思想：（1）优化执行频率较高的基本块；
（2）在循环体内的多个基本块间移动指令，扩大那些执行频率较高的基本块的体积。

- 如何将语句3和5调度到语句2之前？
- 如何保证分支条件为false时结果依然正确？

6.2.1 全局指令调度

第 22 页

- 将上图中的代码转换为下面的MIPS汇编指令

	LD	R4, 0(R1)	// 取A
	LD	R5, 0(R2)	// 取B
	DADDU	R4, R4, R5	// A=A+B
	SD	0(R1), R4	// 存A
	BNEZ	R4, thenpart	// A=0则转移
	X		// 代码段X, 基本块elsepart
	J	join	
thenpart:			// 基本块thenpart
	SD	..., 0(R2)	// 指令I₁ , 对应语句3
join:			
	SD	..., 0(R3)	// 指令I₂ , 对应语句5

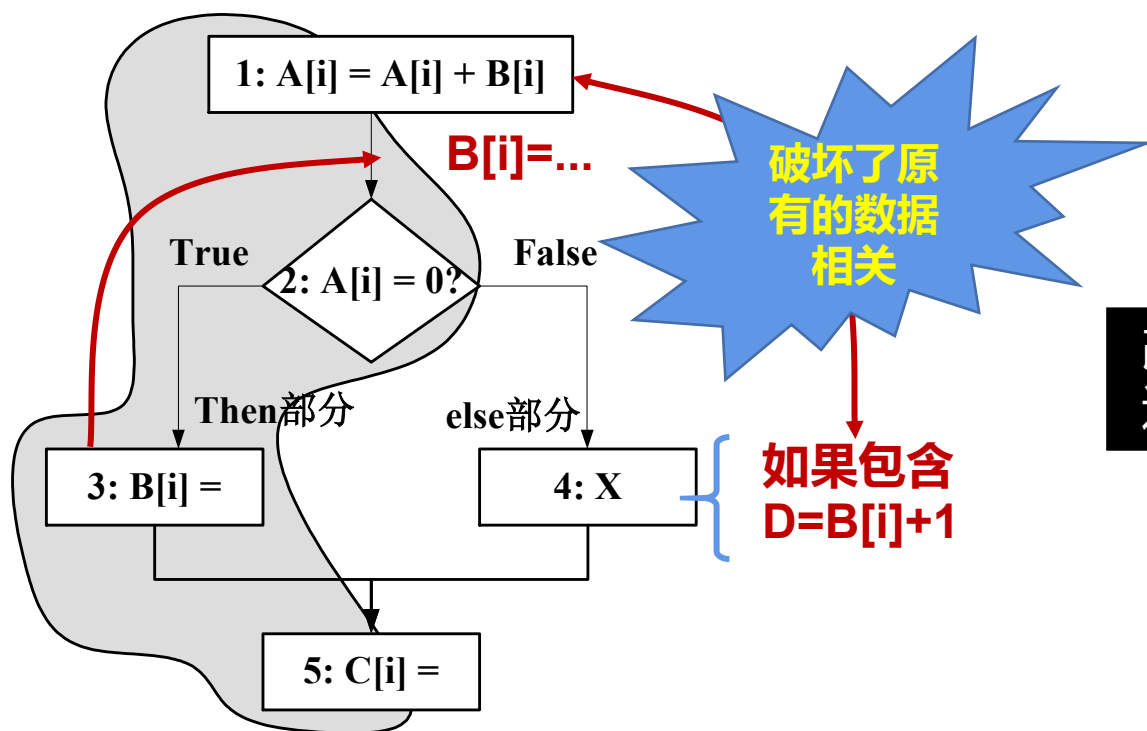
```
graph TD; BNEZ --> thenpart; J --> join; join --> thenpart;
```

6.2.1 全局指令调度

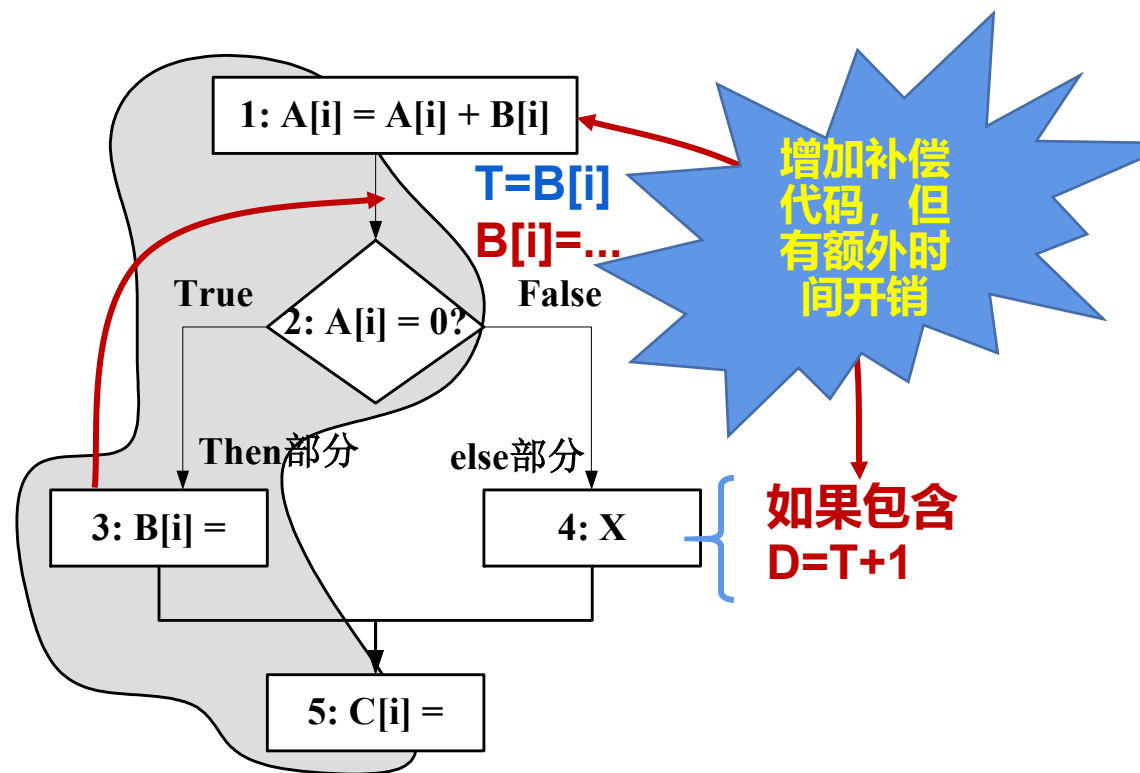
第 23 页

■ 调度指令 I_1 ：直接将 I_1 移到BEQZ前

- 是否会产生错误结果？
- 需不需要向基本块elsepart中增加补偿代码？
- 有没有可能带来额外时间开销？



改进

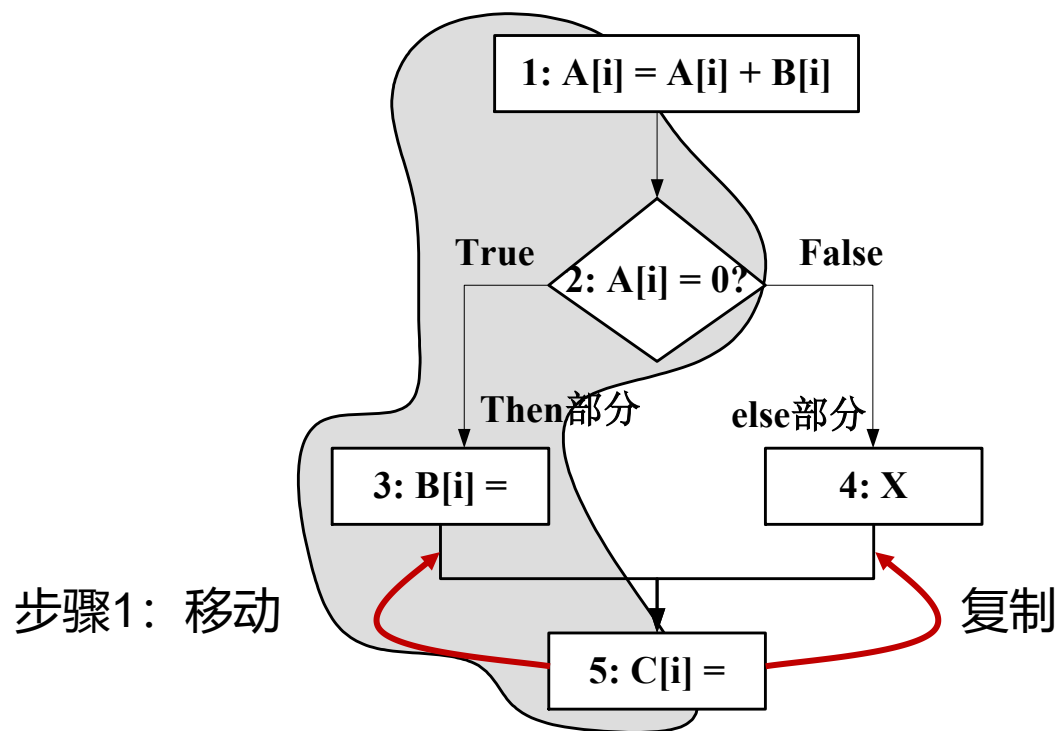


6.2.1 全局指令调度

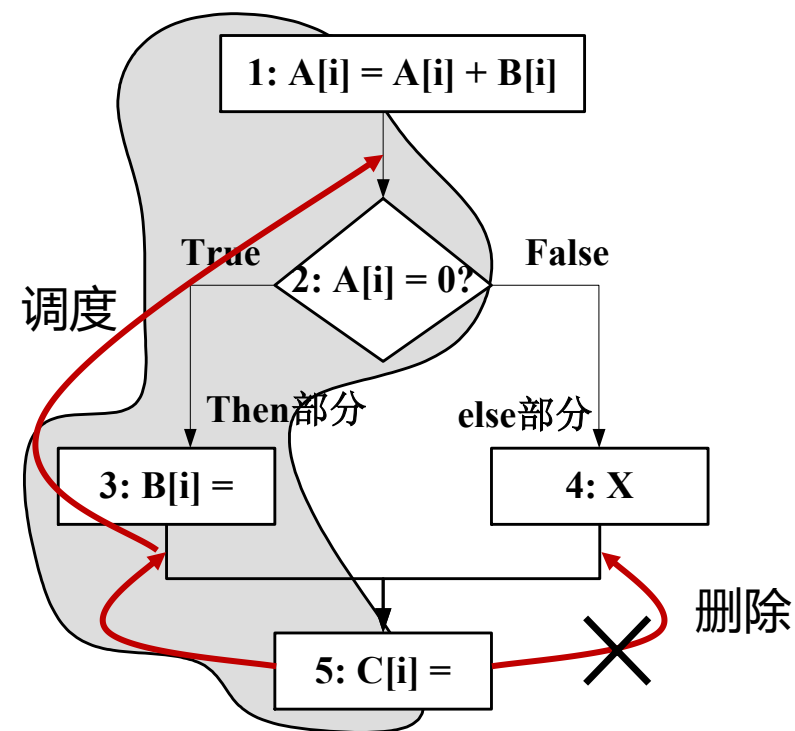
第 24 页

■ 调度指令I₂

- 将I₂移动到基本块thenpart中，同时复制到elsepart中。
- 若不影响执行结果，将I₂调度到BEQZ前，同时删除elsepart中的副本。



步骤2：调度

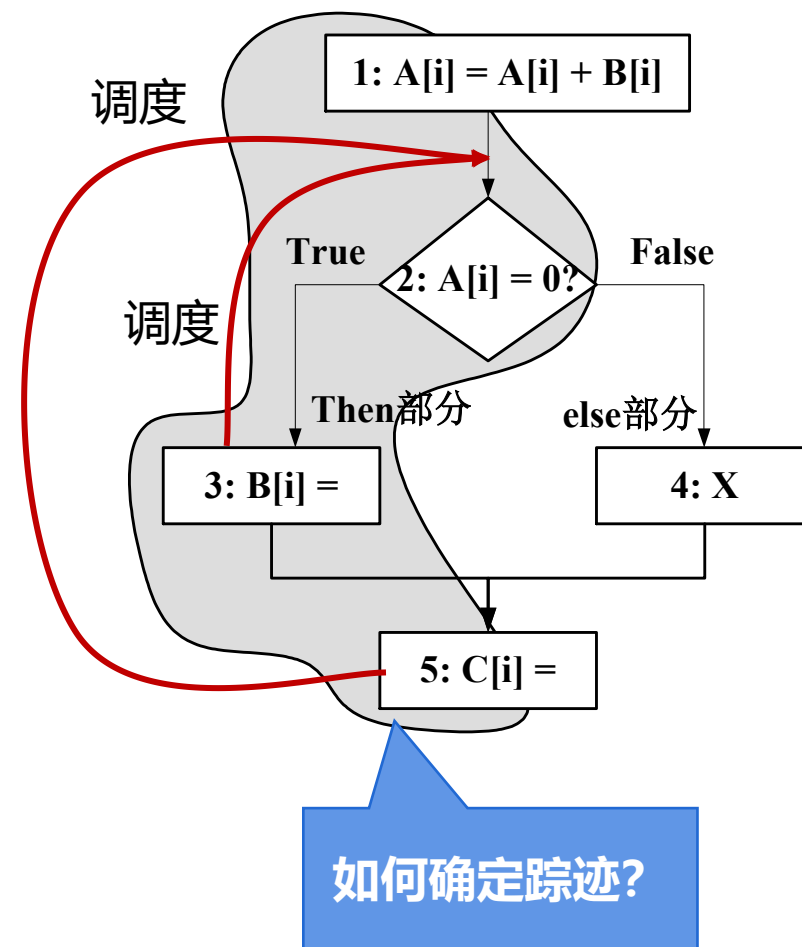


6.2.1 全局指令调度

第 25 页

3. 全局指令调度是一个很复杂的问题，以 I_1 的调度为例：

- 需要确定分支中基本块thenpart和elsepart的执行频率各是多少？
 - 如果thenpart执行频繁，则调度 I_1 可以提升效率，反之可能降低
- 在分支语句前完成 I_1 所需的开销是多大？
 - 如果分支语句前有一些“空转”周期，则 I_1 可以被调度到这样周期内，开销为0
- 调度 I_1 是否能够缩短thenpart块的执行时间？
 - 如果 I_1 是关键路径的第一条语句，则调度该指令可以减少执行开销
- I_1 是否是最佳被调度对象？调度 I_2 或thenpart内的其他指令是否获得更大性能提升。
- 是否需要向elsepart块中增加补偿代码，补偿代码开销如何？怎样生成补偿代码？



1. 概述

- 踪迹 (trace) : 程序执行的指令序列, 通常由一个或多个基本块组成, trace内可以有分支, 但一定不能包含循环。
- 踪迹调度 (trace scheduling) 会优化执行频率高的踪迹, 减少其执行开销。由于存在多个踪迹, 踪迹调度非常适合多流出处理器。
- 由于需要添加补偿代码以确保正确性, 那些执行频率较低的踪迹的开销反而会有所增加, 所以仅当不同踪迹的执行频率差别较大且各条踪迹的执行频率受输入集的影响较小 (补偿代码小) 时, 才能取得较好的效果。

2. 踪迹调度的步骤

分为两步：踪迹选择和踪迹压缩

■ 踪迹选择

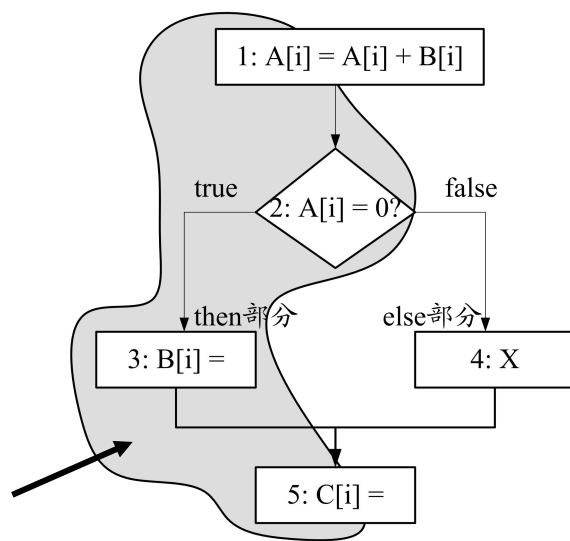
- 作用：从程序的控制流图中选择执行频率较高的路径，每条路径就是一条踪迹；
- 原则：处理转移成功与失败概率相差较大的情况；**相近的情况可以采用谓词执行技术进行优化**；
- 方法一：如果是循环结构，则采用循环展开，一般循环体的执行效率高于循环体外的基本模块的执行效率；
- 方法二：如果是分支结构，则根据典型输入集下的运行统计信息，若转移成功（或失败）的概率高，则视作转移总是成功（或失败）。

6.2.2 踪迹调度

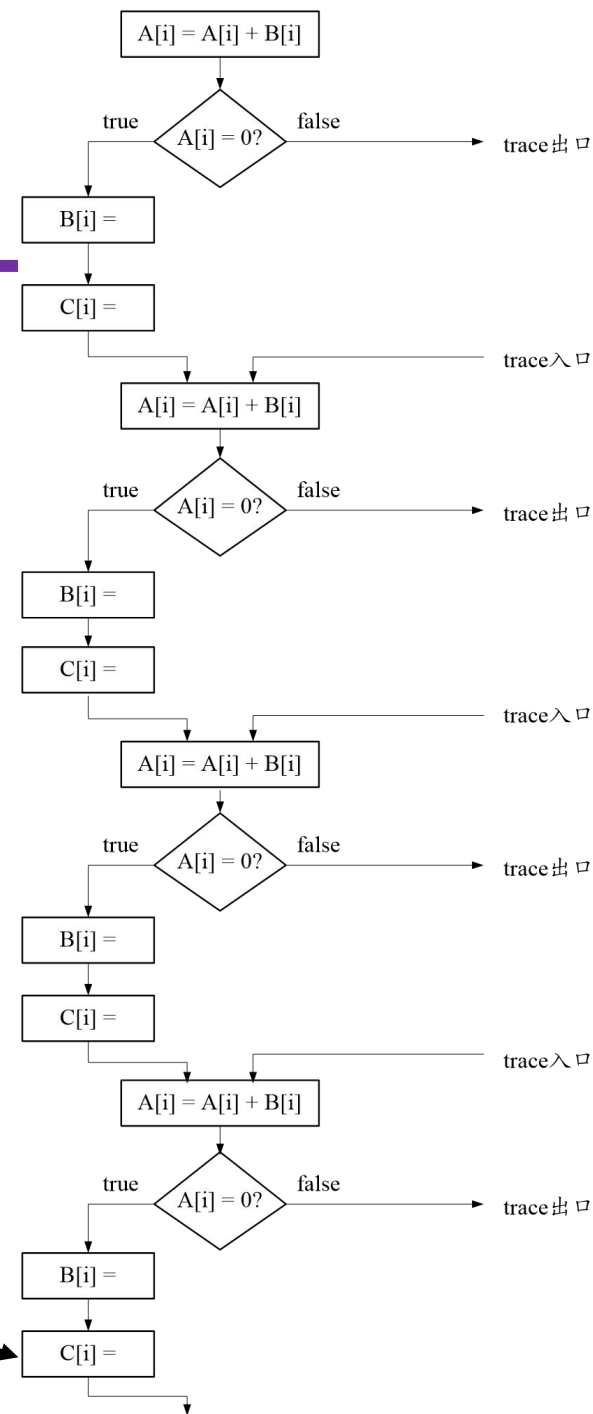
■ 踪迹选择——实例分析

- 将左边的循环展开4次并把阴影部分(执行频率高)拼接在一起就可以得到一条踪迹;
- 一条踪迹可以有多个入口和多个出口。

假设阴影部分的执行频率高



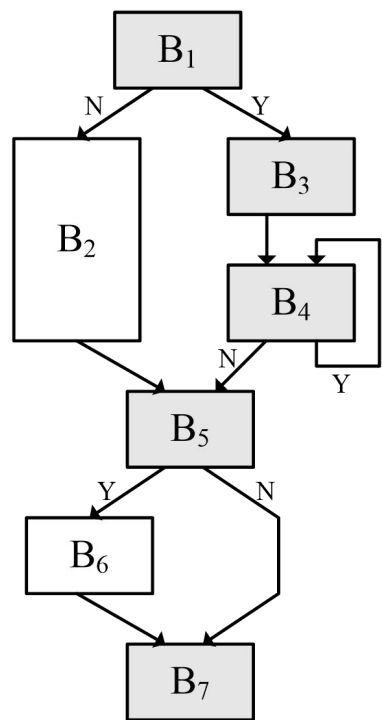
C[i]=语句是该踪迹的一个出口, 执行该语句后控制流将无条件退出该踪迹



6.2.2 踪迹调度

第 29 页

- 踪迹压缩：对已生成的踪迹进行指令调度和优化，尽可能地缩短其执行时间
 - 注意：跨越踪迹内部的入口或出口调度指令时必须非常小心，有时还需要增加补偿代码，确保执行结果的正确性。



(a) 控制流图

假设控制流离开基本块B1和B4后发生转移（分支条件为Y）的概率大，而离开B5后发生转移的概率小

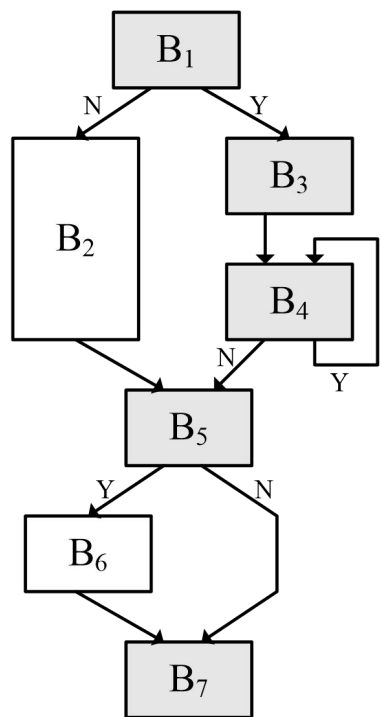
第一步： 经过选择可以确认得到3条踪迹B₁-B₃、B₄以及B₅-B₇

理由：B4是循环体，因此这3条踪迹无法合并在一起。

6.2.2 踪迹调度

第 30 页

- 踪迹压缩：对已生成的踪迹进行指令调度和优化，尽可能地缩短其执行时间
 - 注意：跨越踪迹内部的入口或出口调度指令时必须非常小心，有时还需要增加补偿代码，确保执行结果的正确性。



(a) 控制流图

B₁: $x = x + 1$
 $y = x - y$
if $x < 5$ goto B₃

B₂: $z = x * z$
 $y = y + 1$
goto B₅

B₃: $y = 2 * y$
 $x = x - 2$

压缩前

(b) 踪迹压缩前后的基本块B₁、B₂、B₃

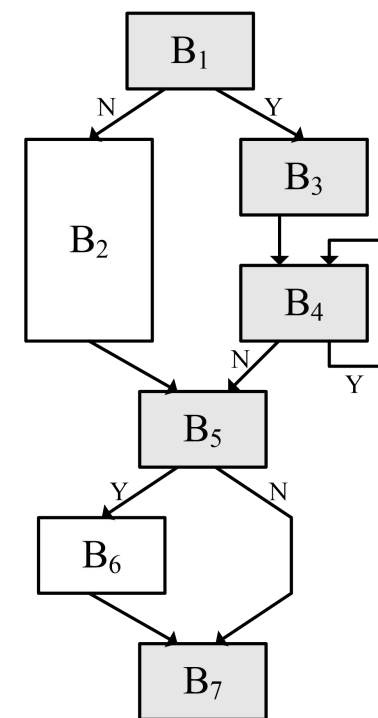
目标：3条踪迹：B₁-B₃、B₄以及B₅-B₇

踪迹压缩：

- (1) 指令 “ $y = x - y$ ” 被从B₁调度到B₃中，跨越了踪迹的一个出口；
- (2) 需要向块B₂中增加补偿代码，即将指令 “ $y = x - y$ ” 复制到B₂的第一条指令之前。

3. 踪迹调度的特点总结

- 踪迹调度能够提升性能的最根本原因在于选出的踪迹 (trace) 都是执行频率很高的路径, 减少它们的执行开销有助于缩短程序的总执行时间。
- 对于某些应用, 补偿代码引起的开销很有可能降低踪迹调度的优化效果。
- 踪迹调度会大大增加编译器的实现复杂度, 特别是指令调度跨跃踪迹的一个入口或出口, 或踪迹的入口或出口在踪迹内部。



(a) 控制流图

如何减少踪迹调度的难度?

1. 概述

- **存在问题**：在踪迹调度中，如果踪迹入口或出口位于踪迹内部，编译器生成补偿代码的难度将大大增加，而且编译器很难评估这些补偿代码究竟会带来多少性能损失。
- **解决方法**：增加对踪迹拓扑的约束，限制只能拥有一个入口
- **定义**：超块（superblock）是只能拥有一个入口，但可以拥有多个出口的结构
- **理由**：由于只能有一个入口，所以只需要考虑跨跃踪迹出口的指令调度
- **接下来的问题**：超块的构造过程与踪迹相似，但怎样确保只有一个入口？

6.2.3 超块调度

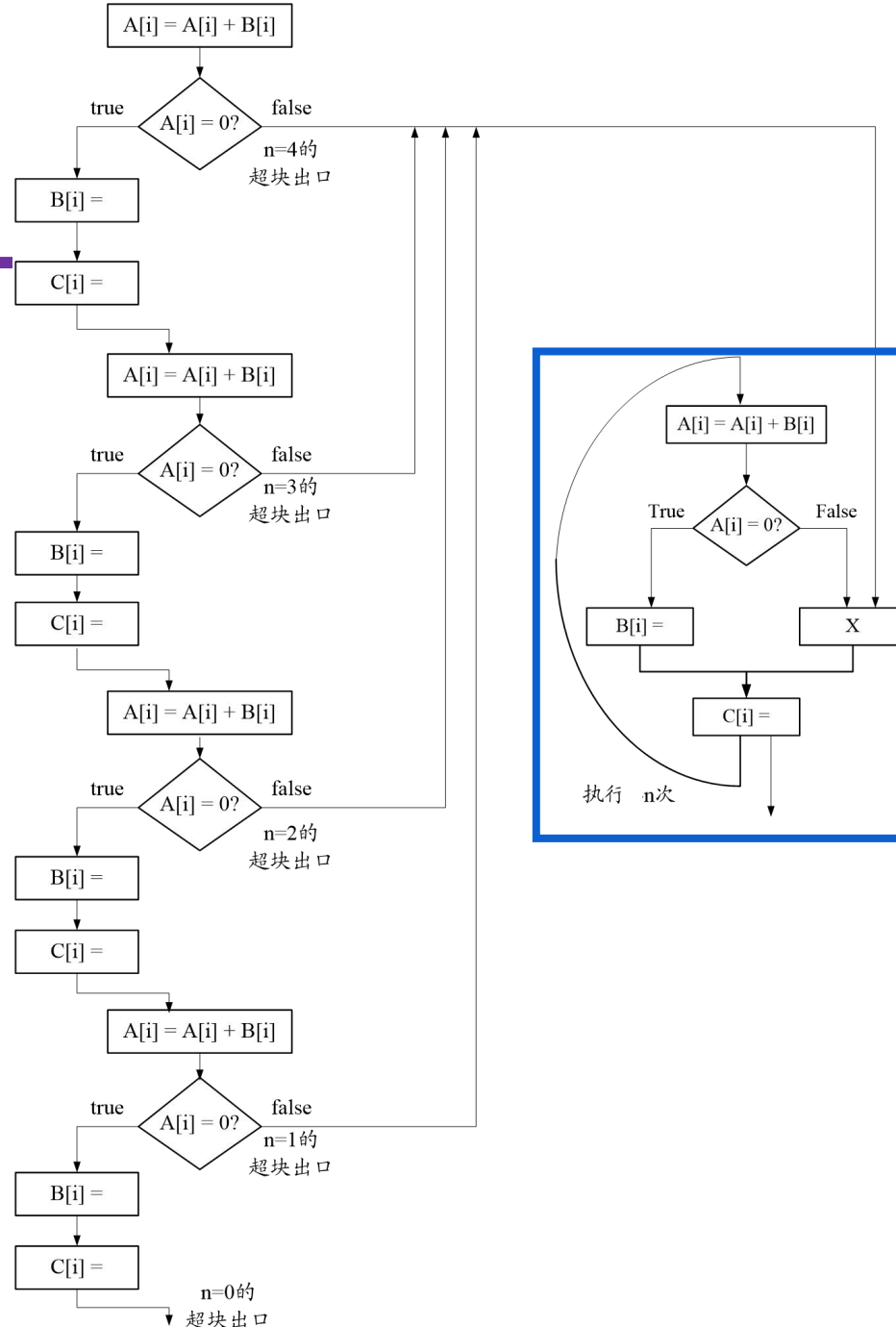
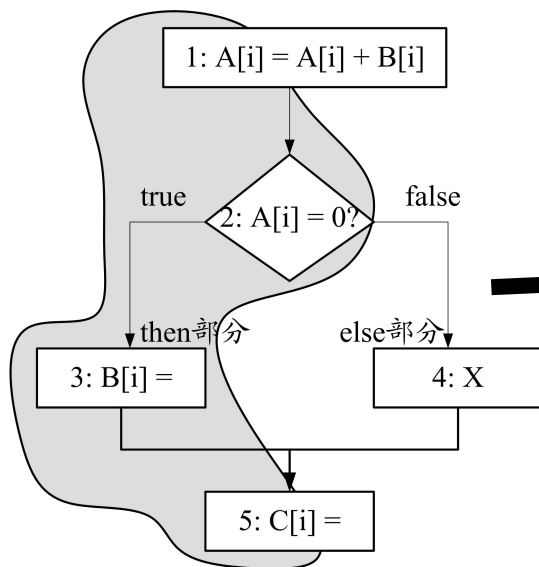
2. 超块构造——尾复制技术

将左边的循环展开4次并把阴影部分(执行频率高)拼接在一起就可以得到一个超块;

超块有1个入口和5个出口($n=4/3/2/1/0$), 表示从该出口离开已经执行了4-n次循环;

除了最后一个出口 $n=0$ 外, 从其他4个出口退出超块后, 还需要继续完成余下的n次叠代(蓝框部分);

蓝框部分复制代码总是作为退出超块后必须执行的补偿代码。



3. 超块调度的特点总结

- 尾复制技术简化了补偿代码的生成过程，并降低了指令调度的复杂度。
- 超块结构目标代码的体积也大大增加。
- 补偿代码的生成使得编译过程更加复杂，而且由于无法准确评估由补偿代码引起的时间开销，这限制方法超块调度的应用范围。

6.3 静态多指令流出：VLIW技术

6.3 静态多指令流出：VLIW技术

第 36 页

1. 基本概念：

把同时流出的或者满足特定约束的一组操作打包在一起，得到一条更长的（64位、128位或更长）的指令。



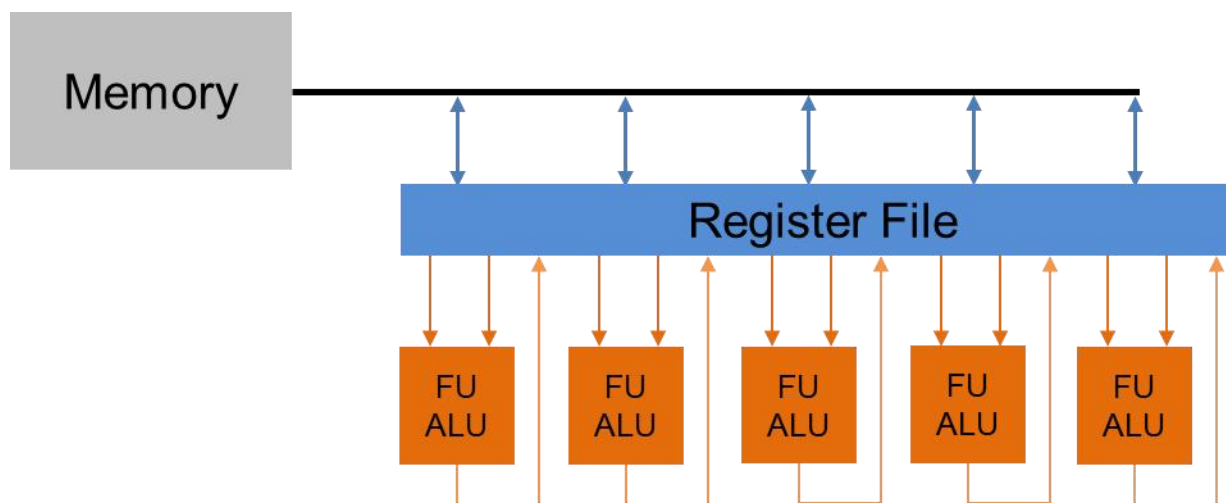
2. 与超标量的对比与分析

- 在动态调度的超标量处理器中，相关检测和指令调度基本都由硬件完成。
- 在静态调度的超标量处理器中，部分相关检测和指令调度工作交由编译器完成。
- 在VLIW处理器中，相关检测和指令调度工作全部由编译器完成，它需要更“智能”的编译器。

6.3 静态多指令流出：VLIW技术

第 37 页

要求：VLIW处理器需要多个功能单元才能支持多个操作同时执行



鉴于相关检测和指令调度工作全部由编译器完成，重点关注分析：

- (1) VLIW处理器的指令发射与执行机制
- (2) VLIW处理器的功能单元利用率

6.3 静态多指令流出：VLIW技术

3. 实例分析

例6.3 假设某VLIW处理器每个时钟周期可以同时流出5个操作，包括2个访存操作，2个浮点操作以及1个整数或分支操作。将例6.1中的代码循环展开，并调度到该VLIW处理器上执行。循环展开次数不定，但至少要是能够保证消除所有流水线“空转”周期，同时不考虑分支延迟。

```
Loop: L.D      F0, 0(R1)
      ADD.D    F4, F0, F2
      S.D      F4, 0(R1)
      DADDIU   R1, R1, #-8
      BNE      R1, R2, Loop
```

产生结果的指令	使用结果的指令	延迟(cycles)
浮点计算	另一个浮点计算	3
浮点计算	浮点store(S.D)	2
浮点Load(L.D)	浮点计算	1
浮点Load(L.D)	浮点store(S.D)	0

6.3 静态多指令流出：VLIW技术

解：循环展开为7次，经调度后可以消除所有流水线“空转”。在不考虑分支延迟的情况下，每执行一个叠代需要9个时钟周期，计算出7个结果，即平均每得到一个结果需要1.29个周期。

访存操作1	访存操作2	浮点操作1	浮点操作2	整数分支操作
L.D F0, 0 (R1)	L.D F6, -8 (R1)	nop	nop	nop

9条指令，最多容纳45个操作，但实际只包含23个操作，其余22个均为空操作，编码效率略大于50%，

6.3 静态多指令流出：VLIW技术

第 40 页

存在问题：

问题一. 编码效率低：如何开发出大量的指令级并级仍是面临的最大挑战；

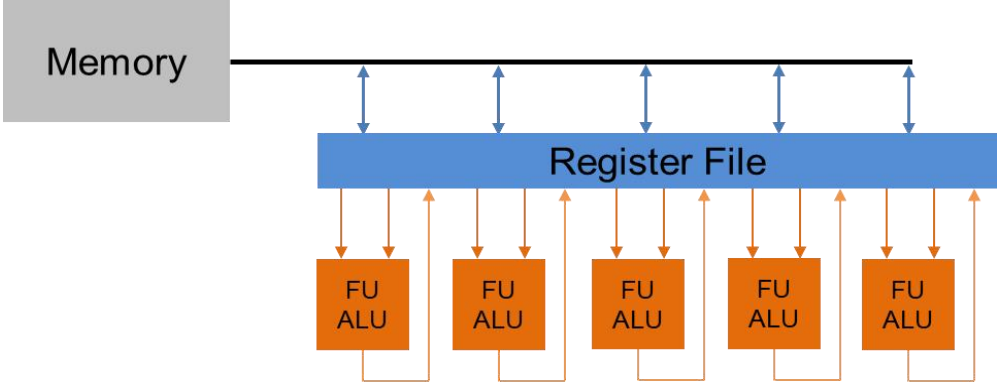
问题二. 缺少相关检测逻辑：为了简化硬件实现，大多数VLIW处理器没有相关检测逻辑，而是靠互锁机制保证执行结果的正确性。

访存操作1	访存操作2	浮点操作1	浮点操作2	整数分支操作
L.D F0, 0 (R1)	L.D F6, -8 (R1)	nop	nop	nop
L.D F10, -16 (R1)	L.D F14, -24 (R1)	nop	nop	nop
L.D F18, -32 (R1)	L.D F22, -40 (R1)	ADD.D F4, F0, F2	ADD.D F8, F6, F2	nop
L.D F26, -48 (R1)	nop	ADD.D F12, F10, F2	ADD.D F16, F14, F2	nop
.....				
S.D 8 (R1) , F28	nop	nop	nop	BNE R1, R2, Loop

由于在编译时无法确定一些操作延迟，所以当一功能单元暂停时互锁机制将暂停整个流水线

6.3 静态多指令流出：VLIW技术

问题三. 目标代码兼容性差：VLIW指令格式与操作类型、功能单元数量以及延迟等体系结构参数密切相关；当这些参数变化指令格式变化。



二进制翻译是一种可行的方法，其原理是将某个硬件平台的二进制代码翻译成另一个平台的目标代码的过程。

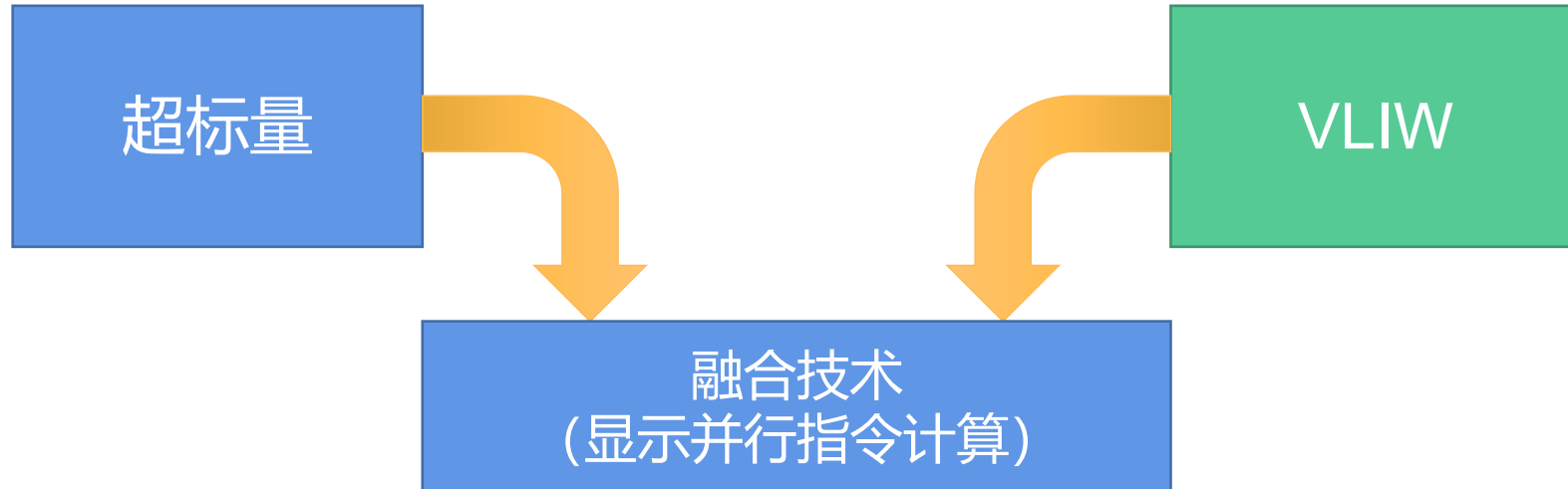
访存操作1	访存操作2	浮点操作1	浮点操作2	整数分支操作
L.D F0, 0 (R1)	L.D F6, -8 (R1)	nop	nop	nop
.....				
L.D F18, -32 (R1)	L.D F22, -40 (R1)	ADD.D F4, F0, F2	ADD.D F8, F6, F2	nop
.....				
S.D 8 (R1) , F28	nop	nop	nop	BNE R1, R2, Loop

6.4 显式并行指令计算EPIC

6.4 显示并行指令计算EPIC

第 43 页

- 超标量和VLIW结构是开发指令级并行的两种极端结构
 - 超标量依赖流水线硬件动态识别可并行指令，存在硬件复杂度太高等问题，8流出成为极限；
 - VLIW依赖编译器实现指令级并行开发，存在代码兼容问题，编译器的智能程度不够。



6.4 显示并行指令计算EPIC

第 44 页

■ EPIC技术在VLIW基础上融合了超标量的一些优点

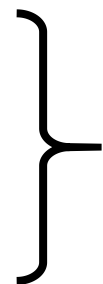
- 编译器根据对程序运行特征的统计信息，如分支指令转移成功的概率，通过踪迹调度、超块调度等带有极强猜测性的优化技术尽可能多地挖掘指令级并行——软件方面
- 流水线硬件提供丰富的计算资源实现这些指令级并行，并通过专门的机制确保在程序执行过程中出现预测错误时仍然能得到正确的运行结果，尽量减少由此引起的额外开销——硬件方面
- 该指令集最重要的思想就是并行处理，如 Itanium和Itanium 2系列。

■ 什么是EPIC?

- 指令级并行主要由编译器负责开发，处理器为保证代码正确执行提供必要的硬件支持，只有在这些硬件机制的辅助下这些优化技术才能高效完成。
- 系统结构必须提供某种通信机制，使得流水线硬件能够了解编译器“安排”好的指令执行顺序。

■ EPIC编译器的高级优化技术

- 非绑定分支
- 谓词执行
- 前瞻执行



系统结构负责设计流水线硬件执行机制，能够了解编译器“安排”好的指令，动态优化执行顺序

6.4.1 非绑定分支

第 46 页

1. 分支指令在传统流水线上的执行过程

- 计算分支转移条件
- 生成分支目标地址
- 取下一条指令
- 译码并流出下一条指令

BNE R1, R2, Loop

在传统流水线上分支指令都具有“原子性”，即上述各操作被绑定在一起，不能分开

2. 非绑定分支技术

- 核心思想：将分支指令划分为多条粒度更小的指令，独立执行。
 - 准备操作：计算分支目标地址
 - 比较操作：计算分支转移条件
 - 转移操作：根据分支转移条件是true还是false，改变控制流或执行顺序的下一条指令。
- 运行时，流水线硬件根据前两个操作的结果，动态地将第三个操作转换为空操作或无条件转移。
 - 前两个操作应尽早完成

```
bne $s0, $s1, EXIT
```

比较\$s0和\$s1两个操作数中的数据，如果不相等则跳转到EXIT指定的地址

- 1、计算EXIT
- 2、比较\$s0和\$s1

如果Y，则bne→nop
如果N，则bne→j EXIT

1. 条件执行机制

- 条件执行：指指令的执行依赖于一定的条件，当条件为真时指令将正常执行，否则将什么也不做。
- 实例：条件传输指令

例6.4 在下面的语句中，

```
if (A=0) {S=T; }
```

假设变量A、S、T的值分别保存在寄存器R1、R2和R3内。请用分支指令和条件传输指令编写功能相同的汇编代码。

6.4.2 谓词执行

第 49 页

解：包含分支指令的MIPS汇编代码如下：

```
BNEZ      R1, L      // A!=0? 执行L
ADDU      R2, R3, R0  // A=0, 则S=T+0
```

L:

而使用条件传输指令CMOVZ时的汇编代码为：

```
CMOVZ R2, R3, R1
```

指令CMOVZ有3个操作数，R2为目的操作数，R1和R3是源操作数，执行条件保存在寄存器R1中。

当R1=0时，R3的值被复制到R2中，否则R2的内容不变。

6.4.2 谓词执行

第 50 页

解：包含分支指令的MIPS汇编代码如下：

```
BNEZ      R1, L      // A!=0? 执行L
ADDU      R2, R3, R0  // S=T+0
```

L:

而使用条件传输指令CMOVZ时的汇编代码为：

```
CMOVZ R2, R3, R1
```



■ 谓词执行 (predicated execution)

- 给指令集中的每条指令都增加一个执行条件，这个执行条件就叫做谓词 (predicate)。
- 若谓词为真，指令正常执行，否则什么也不做。

2. 条件传输指令的应用——计算绝对值

- 求绝对值的运算 $A = \text{abs}(B)$ ，对应的C语句为：

`if (B < 0) {A = -B;} else {A = B;}`

- 使用条件传输指令后，代码段如下（假设变量A和B分别被保存在寄存器R1和R2中）：

`SUB R1, R0, R2` `// A = -B`

`SLT R3, R2, R0` `// 若B < 0, R3 = 1, 否则R3 = 0`

`CMOVZ R1, R2, R3` `// R3 = 1时, A = -B; R3 = 0时, A = B`

3. 谓词执行机制

■ 条件传输指令的性能问题

- 条件传输是最简单的条件执行机制，当一个分支结构的then部分和else部分中仅含有少量指令时，效果较好；

if (B < 0) {A = -B;} else {A = B;}

- 但随着指令数的增加，经过if转换得到的条件传输指令和条件计算指令的数量也将增加，大大降低目标代码的效率。

4. 谓词执行机制——实例分析

例6.5 假设在一个周期内，某双流出的超标量处理器可以同时执行一个访存操作和一个ALU操作，或者仅执行一个分支操作。受此限制，下面这段汇编代码的执行效率并不高，表现在：

（1）第二个周期只能流出一条ALU指令，访存单元空闲；（2）当分支转移不成功时，BEQZ指令后的两条LW指令之间存在的数据相关将引起流水线暂停。

问：试通过谓词执行机制解决这两个问题，减少此段代码的执行开销。

周期	指令1	指令2
1	LW R1, 40 (R2)	ADD R3, R4, R5
2		ADD R6, R3, R7
3	BEQZ R10, L	
4	LW R8, 20 (R10)	
5	LW R9, 0 (R8)	

6.4.2 谓词执行

解：我们用LWC表示带谓词的LW指令，并假设该指令的执行条件为谓词不等于0。这样，BEQZ后的第一条LW指令就可以被转换为LWC指令，并被调度到第二个周期执行，如下所示：

周期	指令1	指令2
1	LW R1, 40 (R2)	ADD R3, R4, R5
2	LWC R8, 20 (R10) , R10	ADD R6, R3, R7
3	BEQZ R10, L	
4	LW R9, 0 (R8)	



6.4.2 谓词执行

第 55 页

■ 分析

- 调度后代码的执行时间缩短了 **(5个周期->4个周期)**
- 若分支转移成功，LWC将被转换为空操作，这不影响结果的正确性，但也不会缩短执行时间；

周期	指令1	指令2
1	LW R1, 40 (R2)	ADD R3, R4, R5
2	LWC R8, 20 (R1), R10	ADD R6, R3, R7
3	BEQZ R10, L (成功)	
4	LW R9, 0 (R8)	

6.4.2 谓词执行

第 56 页

■ 分析

- 调度后代码的执行时间缩短了 **(5个周期->4个周期)**
- 若分支转移成功，LWC将被转换为空操作，这不影响结果的正确性，但也不会缩短执行时间；若分支转移不成功，减少数据相关的影响。
- 周期4中的LW指令转换为LWC，结果如何？

周期	指令1	指令2
1	LW R1, 40 (R2)	ADD R3, R4, R5
2	LWC R8, 20 (R10) , R10	ADD R6, R3, R7
3	BEQZ R10, L (不成功)	
4	LW R9, 0 (R8) -->LWC	

5. 谓词执行机制——异常处理

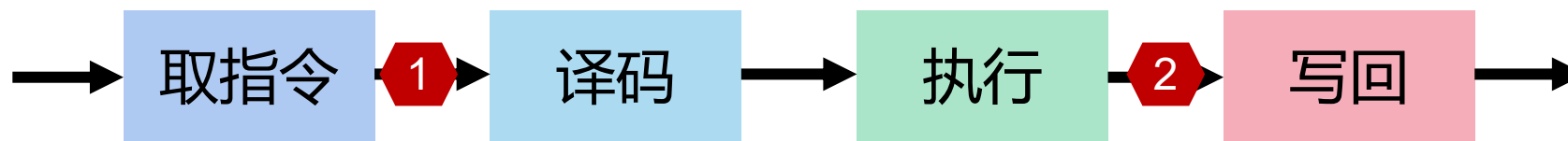
- 谓词执行增加了异常处理的复杂度
 - 例若LWC指令执行时发生缺页中断，如何处理？
 - LWC的谓词为true，中断本应发生；
 - LWC的谓词为false，中断不应发生。
- } 异常处理机制需要根据LWC指令谓词的值决定是否处理这个异常

周期	指令1	指令2
1	LW R1, 40 (R2)	ADD R3, R4, R5
2	LWC R8, 20 (R10) , R10 	ADD R6, R3, R7
3	BEQZ R10, L	
4	LW R9, 0 (R8)	

6.4.2 谓词执行

第 58 页

6. 如何将谓词为假的指令转换为空操作?



■ 两种方法

- 流水线前端指令流出时，必须提早知道谓词的值（分支转移条件值），但很难。谓词指令与指令分支条件的指令之间存在数据相关引发“空转”
- 流水线后端结果确认时，流水线做无用功，降低了功能单元的实际利用率

■ 一般采用第二种方法

- 为什么？

周期	指令1	指令2
1	LW R1, 40 (R2)	ADD R3, R4, R5
2	LWC R8, 20 (R10) , R10	ADD R6, R3, R7

需要足够多的指令级并行填满所有新增的指令槽 (NOP)



1. 概述

■ 谓词执行与全局指令调度的不足之处

- 仅有少量结构全面实现谓词执行，如IA-64；部分实现谓词执行，如MIPS等
- 全局指令调度往往需要补偿代码，降低了猜测执行的性能收益

■ EPIC通过前瞻执行提高猜测执行的效果，前瞻执行的定义：

- 在数据相关或控制相关尚未消除时，将指令调度到相关指令前猜测执行，同时还通过硬件机制完成异常处理，确保正确性。
- 影响前瞻执行效果的因素：
 - ✓ 编译器能力的高低：能否准确识别可以被前瞻执行的指令。
 - ✓ 异常处理机制：能否推迟处理由被前瞻执行的指令引起的异常，直到确定前瞻指令确实被执行后。
 - ✓ 如何避免前瞻引起的错误？例如LOAD指令调度在STORE指令之前，且地址相同

2. 实例分析

例6.6 下面是一个if-then-else结构的C程序段以及相应的MIPS汇编代码段，其中变量A和B分别被保存在地址为0(R3)和0(R2)的存储单元中。若分支转移不成功的概率很大，请利用前瞻执行技术将第二条LD指令调度到分支指令BNEZ前执行。假设寄存器R14空闲。

C语句：
if (A≠0) A=A+4; else A=B;



汇编指令：

LD	R1, 0 (R3)	// 取A
BNEZ	R1, L1	// (A≠0) ?
LD	R1, 0 (R2)	// A=B (else部分)
J	L2	
L1: DADDI	R1, R1, #4	// A=A+4(then部分)
L2: SD	R1, 0 (R3)	// 存A

6.4.3 前瞻执行

第 61 页

解：调度结果如下：

LD	R1, 0 (R3)	// 取A
sLD	R14, 0 (R2)	// 取B, 前瞻执行
BEQZ	R1, L3	
DADDI	R14, R1, #4	// A=A+4
L3: SD	R14, 0 (R3)	// A=B

数据流发生变化?

C语句:
if (A≠0) A=A+4; else A=B;

问题：若执行sLD时发生异常应该如何处理？

两类异常：

- **终止性异常：**异常发生时程序执行将被迫终止，如违反存储保护权限异常
- **可继续异常：**异常处理后程序仍可继续执行，如缺页

重点解决

3. 前瞻执行——异常处理的4种方法

■ 方法一： 立即处理

- 此前瞻指令引起的异常只是简单地返回一个未定义值即可，而不是立即结束程序的运行
- 前瞻正确时，正在执行的程序不会被终止，但它的执行结果肯定是错误的
- 前瞻错误时，程序也将继续执行下去，只是处理该异常的返回值不会被使用
- 少数处理器提供快速异常处理模式

6.4.3 前瞻执行

第 63 页

■方法二：借助专门的检测指令判断是否需要处理

➤ 代码实例

```
LD          R1, 0 (R3)          // 取A
sLD         R14, 0 (R2)         // 取B, 前瞻执行
BEQZ        R1, L1
SPECCK      0 (R2)              // sLD是否产生异常
J           L2
L1: DADDI    R14, R1, #4         // A=A+4
L2: SD       R14, 0 (R3)        // A=B
```

➤ SPECCK的执行条件与sLD指令相同（分支转移失败）

➤ 基本思想：推迟处理由前瞻指令sLD引发的异常，直到已确定该指令确实应该被执行。

6.4.3 前瞻执行

第 64 页

■方法三：借助寄存器状态位判断是否需要处理

- 为每个通用寄存器增加一个特殊的状态标志位：“poison” 位
- 前瞻指令引发的可继续异常都将被立即处理
- 前瞻指令引发终止性异常时，其目的寄存器R的poison位将被置1，否则该位将被清0
- 当前瞻指令之后的另一条指令访问R时，若R的poison位为1将触发一个终止性异常
- 基本思想：将前瞻指令引起异常的处理推迟到另一条指令访问前瞻指令的目的寄存器时

■方法四：借助再定序缓冲器完成

- 将指令的执行结果保存在再定序缓冲器内，并按指令流出的顺序依次确认。
- 前瞻指令的确认时机被推迟，直至能够确定该指令的前瞻执行是正确（或错误）的。
- 除了需要再定序缓冲器等硬件机制的支持外，也需要在编译时标出所有被前瞻的指令，以及这些指令所跨越的条件分支。

4. 控制前瞻

- 将load调度到store之前前瞻执行最常见的数据前瞻
 - 为保证正确性，编译器总是会选择保守的调度方法，认为相邻的store与load间存在地址冲突，但在很多情况下，地址并不冲突。
 - 尽早完成load指令有助于缩短关键路径的长度。

周期	指令1	指令2
1	LW R1, 40 (R2)	ADD R3, R4, R5
2	LWC R8, 20 (R10) , R10	ADD R6, R3, R7
3	BEQZ R10, L	
4	SW R9, 0 (R8)	

4. 控制前瞻

■ 硬件地址冲突检测机制

- 当load指令前瞻执行时，流水线硬件会将它访存的地址记录在一个特殊的地址表中。
- 每执行一条store指令，流水线硬件将该指令的访存地址与地址表中的各有效项进行匹配，命中则说明出现地址冲突，前瞻失败。
- 若控制流抵达load指令原来所在的位置时未出现冲突（编译时在此位置放置检测指令），说明前瞻成功，流水线硬件从地址表中删除对应的项。

周期	指令1	指令2
1	LW R1, 40 (R2)	ADD R3, R4, R5
2	LWC R8, 20 (R10) , R10	ADD R6, R3, R7
3	BEQZ R10, L	
4	SW R9, 0 (R8)	

6.4.3 前瞻执行

第 68 页

■ 数据前瞻失败时的处理

- 仅load指令被前瞻执行：由检测指令重新执行load指令即可
- 数据相关于load的其他指令也被前瞻执行：需要补偿代码

汇编指令：

LD	R1, 0 (R3)	// 取A
sLD	R14, 0 (R2)	// A=B (else部分)
BNEZ	R1, L1	// (A≠0) ?
LD	R14, 0 (R2)	// A=B (else部分)
J	L2	
L1: DADDI	R14, R1, #4	// A=A+4(then部分)
L2: SD	R14, 0 (R3)	// 存A

重新执行

6.5 开发更多的指令级并行

6.5 开发更多的指令并行

第 70 页

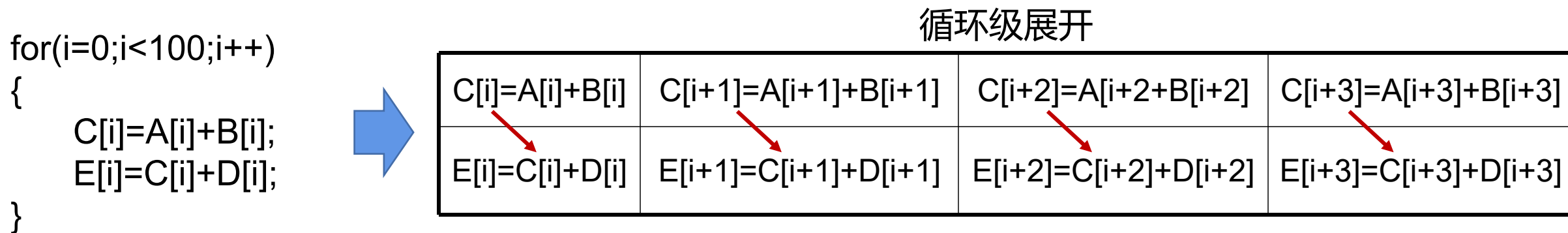
- 全局指令调度技术已经能够很好地处理由分支指令引起的控制相关
 - 容易预测的——前瞻执行
 - 不容易预测的——谓词执行
- 指令间的数据相关对指令级并行开发的限制作用反而越来越大

仍面向循环结构讨论处理数据相关的编译优化技术

6.5 开发更多的指令并行

第 71 页

循环级并行 (Loop-Level Parallelism)：循环中不同迭代之间的并行。由于每个循环中含有多条指令，所以它的粒度比指令级并行大。



分析：从一个循环中开发出多少并行受到的制约因素

- 迭代内部相关：同一循环迭代内部的指令相关，这属于循环体内部的基本指令调度问题；
- 结论：迭代内各语句间的数据相关对于各次迭代是否能够并行没有影响
- 条件：只要保证同一迭代内存在数据相关的各语句之间的相对顺序不变，多个循环迭代就可以并行执行。

6.5.1 挖掘更多的循环级并行

第 72 页

1. 循环携带相关

- 循环携带相关是指一个循环的某个迭代中的指令与其他迭代中的指令之间的数据相关，会大大限制循环展开的效果。

例6.7 在下面的循环中，

```
for (i=1; i<=100; i=i+1) {  
    A[i+1] = A[i] + C[i];           /* S1 */  
    B[i+1] = B[i] + A[i+1];        /* S2 */  
}
```

假设数组A、B和C中所有元素的存储地址都互不相同，请问语句S1与S2之间存在哪些数据相关？

6.5.1 挖掘更多的循环级并行

第 73 页

解 S1和S2之间存在两种不同类型的数据相关：

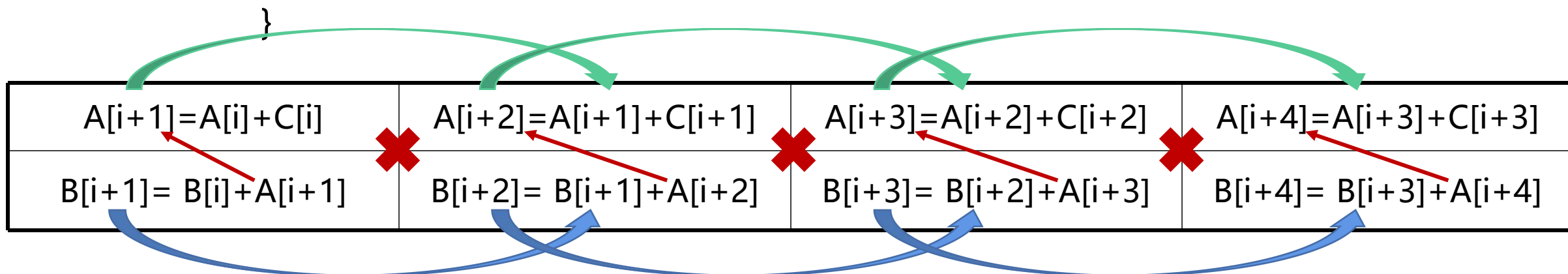
```
for (i=1; i<=100; i=i+2) {  
    A[i+1] = A[i] + C[i];           /* S1 */  
    B[i+1] = B[i] + A[i+1];         /* S2 */  
    A[i+2] = A[i+1] + C[i+1];       /* S3 */  
    B[i+2] = B[i+1] + A[i+2];       /* S4 */  
}
```

- RAW数据相关：同一叠代内的语句S2与S1之间；
- 循环携带RAW数据相关：相邻连词叠代的语句S1之间，相邻两次叠代中的语句S2之间。

6.5.1 挖掘更多的循环级并行

第 74 页

```
for (i=1; i<=100; i=i+1) {  
    A[i+1] = A[i] + C[i];           /* S1 */  
    B[i+1] = B[i] + A[i+1];        /* S2 */  
}
```



分析:

- 循环携带相关迫使指令只能按照所在叠代的先后顺序依次执行。
- 限制了同一叠代内存在数据相关的各语句之间的相对顺序。

6.5.1 挖掘更多的循环级并行

第 75 页

■ 怎样消除循环携带数据相关？

例6.8 在下面的循环中，语句S1和S2之间存在哪些数据相关？该循环的各次迭代是否可以并行执行？如果不能，请修改其代码，使之可以并行。

```
for (i=1; i<=100; i=i+1) {  
    A[i]    = A[i] + B[i];          /* S1 */  
    B[i+1] = C[i] + D[i];          /* S2 */  
}
```

$A[i] = A[i] + B[i]$	$A[i+1] = A[i+1] + B[i+1]$	$A[i+2] = A[i+2] + B[i+2]$	$A[i+3] = A[i+3] + B[i+3]$
$B[i+1] = C[i] + D[i]$	$B[i+2] = C[i+1] + D[i+1]$	$B[i+3] = C[i+2] + D[i+2]$	$B[i+4] = C[i+3] + D[i+3]$

6.5.1 挖掘更多的循环级并行

第 76 页

解 第*i*次叠代中语句S1与第*i*-1次叠代中语句S2之间存在RAW类型的循环携带数据相关，但它们之间没有形成环(S2与上次叠代的S1不相关)。修改后代码如下：

```
A[1] = A[1] + B[1];  
for (i=1; i<=99; i=i+1) {  
    B[i+1] = C[i] + D[i];           /* 原S2 */  
    A[i+1] = A[i+1] + B[i+1];       /* 原S1 */  
}  
B[101] = C[100] + D[100];
```

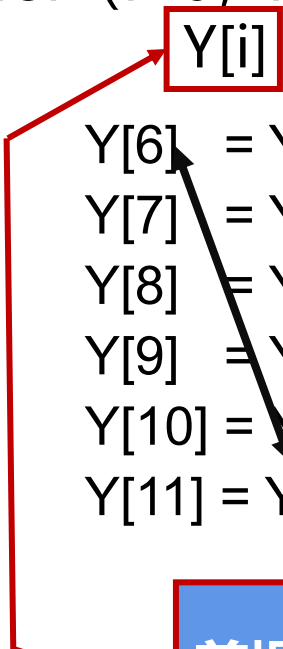
$B[i+1] = C[i] + D[i]$	$B[i+2] = C[i+1] + D[i+1]$	$B[i+3] = C[i+2] + D[i+2]$	$B[i+4] = C[i+3] + D[i+3]$
$A[i+1] = A[i+1] + B[i+1]$	$A[i+2] = A[i+2] + B[i+2]$	$A[i+3] = A[i+3] + B[i+3]$	$A[i+4] = A[i+4] + B[i+4]$

6.5.1 挖掘更多的循环级并行

第 77 页

■ 复杂循环携带数据相关的处理

```
for (i=6; i<=100; i=i+1)
    Y[i] = Y[i-5] + Y[i]; // 相关距离为5
```



$Y[6] = Y[1] + Y[6]$
 $Y[7] = Y[2] + Y[7]$
 $Y[8] = Y[3] + Y[8]$
 $Y[9] = Y[4] + Y[9]$
 $Y[10] = Y[5] + Y[10]$
 $Y[11] = Y[6] + Y[11]$

```
for (i=2; i<=100; i=i+1)
    Y[i] = Y[i-1] + Y[i]; // 相关距离为1
```

$Y[2] = Y[1] + Y[2]$
 $Y[3] = Y[2] + Y[3]$

相关距离越大，通过展开循环可以获得的并行度越高！

前提：编译器知道相关距离

问题：编译器如何知道 $Y[i]$ 和 $Y[i-5]$ 是指向同一个数据？

2. 存储别名分析

■ 什么是存储别名

➤ 一个元素可能同时拥有多个合法的地址表达式

➤ $A[i+5]$ 、 $A[j*2-6]$ 、 $\&A[k]$

```
for (i=6; i<=100; i=i+1)
```

```
    Y[i] = Y[i-5] + Y[i]; // 相关距离为5
```

```
                        // 相关距离为1
```

■ 存在的问题：

➤ 由于缺少足够的运行信息（如不知道索引变量 i 、 j 、 k 的值），编译时很难确定这三个表达式的值是否相同，则无法确定这三条指令是否相关。为了确保正确性，编译器通常会保守地假设它们将访问同一元素，放弃进行优化调度机会。

2. 存储别名分析

■ 数组是仿射的

```
for (i=6; i<=100; i=i+1)  
    Y[i] = Y[i-5] + Y[i];
```

- 如果一个一维数组 $A[m:n]$ 的下标可以被表示为形如 $a \times i + b$ 的形式，那么就称该数组是仿射的（affine）。 i 是循环索引变量，而 m 和 n 分别表示 i 取值的下界和上界。
- 一个多维数组，如果它每一维的下标都是仿射的，那么它就是仿射的。

问题：判断访问数组 $A[m:n]$ 的两条指令是否相关的问题就可以转换为判断这两条指令所访问的数组元素是否是具有相同的下标

6.5.1 挖掘更多的循环级并行

第 80 页

■ GCD测试法

- 假设用下标 $a \times j + b$ 将数组A的一个元素写入存储器，而用下标 $c \times k + d$ 将数组A的一个元素从存储器中读出，这里j和k都是循环A的同一元素， $m \leq j$ ， $k \leq n$ 。显然，当且仅当 $a \times j + b = c \times k + d$ 时，这两条指令将会访问数组A的同一元素。

➤ 算法描述

- ✓ 假设a、b、c、d的值均已知

- ✓ 如果 $\text{GCD}(c, a)$ 可以整除 $(d-b)$ ，那么有可能存在存储别名。

- ✓ 如果GCD测试的结果为假（不能整除），那么一定没有存储别名存在。

假设 $j=k$ ，则 $a \times j - c \times k = d - b$

$\rightarrow \text{GCD}(c, a) \times g = d - b$

$\frac{d - b}{\text{GCD}(c, a)} = g$ 如果g是整数，则可能存在；则不可能

知识点：GCD(a,b)是求a和b的最大公约数,得到的最大公约数

6.5.1 挖掘更多的循环级并行

第 81 页

例6.9 使用GCD测试方法判断下面的循环中是否存在存储别名。

```
for (i=2; i<=100; i=i+1)
    x[2*i+3] = x[2*i] * 5.0;
```

解 在这个循环中, $a=2$, $b=3$, $c=2$, $d=0$,

那么 $\text{GCD}(a, c) = \text{GCD}(2, 2) = 2$, 而 $d-b = 0-3 = -3$ 。

由于2不能整除-3, 因此没有存储别名, 即无论*i*取何值, $x[2*i+3]$ 与 $x[2*i]$ 都将表示数组*x*的不同元素。

i=2		i=3		i=4		i=5		i=6	
x[7]	x[4]	x[9]	x[6]	x[11]	x[8]	x[13]	x[10]	x[15]	x[12]

6.5.1 挖掘更多的循环级并行

第 82 页

例6.9 使用GCD测试方法判断下面的循环中是否存在存储别名。

```
for (i=2; i<=100; i=i+1)
    x[2*i+3] = x[2*i-1] * 5.0;
```

解 在这个循环中, $a=2$, $b=3$, $c=2$, $d=-1$,

那么 $\text{GCD}(a, c) = \text{GCD}(2, 2) = 2$, 而 $d-b = -1-3 = -4$ 。

由于2能整除-4, 因此有可能存在存储别名, 存在 i 取某值, $x[2*i+3]$ 与 $x[2*i-1]$ 将指向相同的元素。 **(相关距离=2)**

i=2		i=3		i=4		i=5		i=6	
x[7]	x[3]	x[9]	x[5]	x[11]	x[7]	x[13]	x[9]	x[15]	x[11]

3. 数据相关分析

- 除了检测指令之间是否存在数据相关外，编译器还会将识别出的数据相关进一步细分为**真数据相关、输出相关和反相关**等不同类型，以便利用不同的优化技术消除这些相关。
- 常用的优化有**重命名、值传播、高度消减**等。

6.5.1 挖掘更多的循环级并行

第 84 页

■ 重命名优化实例

例6.10 找出下面循环中的所有数据相关，指出它们究竟是真数据相关、输出相关、还是反相关，并利用重命名技术消除其中的输出相关和反相关。

```
for (i=1; i<=100; i=i+1) {  
    Y[i] = X[i] / a;          /* S1 */  
    X[i] = X[i] + a;          /* S2 */  
    Z[i] = Y[i] + a;          /* S3 */  
    Y[i] = a - Y[i];          /* S4 */  
}
```

6.5.1 挖掘更多的循环级并行

第 85 页

解 这4条语句中存在以下相关：

```
for (i=1; i<=100; i=i+1) {  
    Y[i] = X[i] / a;    /* S1 */  
    X[i] = X[i] + a;    /* S2 */  
    Z[i] = Y[i] + a;    /* S3 */  
    Y[i] = a - Y[i];    /* S4 */ }
```



1. S3与S1和S4与S1之间分别存在真数据相关
2. S2和S1之间存在反相关
3. S3和S4之间存在反相关
4. S1和S4之间存在输出相关

6.5.1 挖掘更多的循环级并行

第 86 页

将原代码变换为下面的形式，可以消除所有输出相关和反相关。

```
for (i=1; i<=100; i=i+1) {  
    /* 将数组Y重命名为T以消除输出相关 */  
    T[i] = X[i] / a;  
    /* 将数组X重命名为X1以消除反相关 */  
    X1[i] = X[i] + a;  
    /* 将Y重命名为T以消除反相关 */  
    Z[i] = T[i] + a;  
    Y[i] = c - T[i]; }  
}
```

影响仅
限于循
环体内

编译器应将循环体后的所有语句中的X替换为X1

6.5.1 挖掘更多的循环级并行

第 87 页

■ 值传播优化实例

- 原理：值传播优化通过将变量替换为已知的值或表达式以达到消除数据相关
- 实例：

优化前代码：

```
DADDUI    R1, R2, #4
```

```
DADDUI    R1, R1, #4
```

优化后代码：

```
DADDUI    R1, R2, #8
```

6.5.1 挖掘更多的循环级并行

第 88 页

■ 高度消减优化实例

- 目的：对于多流出处理器，简单地减少指令数量不一定能够真正缩短执行时间，只胡缩短这些指令在数据流图中的关键路径长度才能提升性能

- 优化前代码I

```
ADD  R1, R2, R3 /* I1 */  
ADD  R4, R1, R6 /* I2 */  
ADD  R8, R4, R7 /* I3 */
```

真数据相关
真数据相关

关键路径长度=3

- 优化后代码I

```
ADD  R1, R2, R3  
ADD  R4, R6, R7  
ADD  R8, R1, R4
```

真数据相关

关键路径长度=2

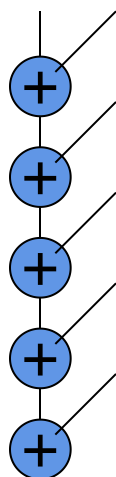
6.5.1 挖掘更多的循环级并行

第 89 页

■ 高度消减优化实例: $\text{sum} = \text{sum} + X[i]$ /* i 为循环索引变量 */

➤ 优化前代码 II

```
sum = sum + X[i];  
sum = sum + X[i+1];  
sum = sum + X[i+2];  
sum = sum + X[i+3];  
sum = sum + X[i+4];
```

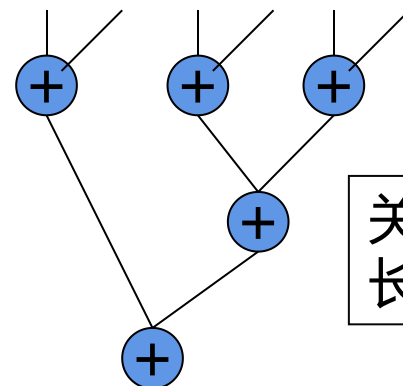


关键路径
长度为5



➤ 优化后代码 II

```
sum = sum + X[i];  
t1 = X[i+1] + X[i+2];  
t2 = X[i+3] + X[i+4];  
t1 = t1 + t2;  
sum = sum + t1;
```

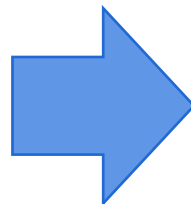


关键路径
长度为3

6.5.2 软流水

第 90 页

```
for (i=1; i<=100; i=i+1) {  
    A[i]    = A[i] + B[i];          /* S1 */  
    B[i+1] = C[i] + D[i];          /* S2 */  
}
```



```
A[1] = A[1] + B[1];  
for (i=1; i<=99; i=i+1) {  
    B[i+1] = C[i] + D[i];          /* 原S2 */  
    A[i+1] = A[i+1] + B[i+1];      /* 原S1 */  
}  
B[101] = C[100] + D[100];
```

$A[i] = A[i] + B[i]$	$A[i+1] = A[i+1] + B[i+1]$	$A[i+2] = A[i+2] + B[i+2]$	$A[i+3] = A[i+3] + B[i+3]$
$B[i+1] = C[i] + D[i]$	$B[i+2] = C[i+1] + D[i+1]$	$B[i+3] = C[i+2] + D[i+2]$	$B[i+4] = C[i+3] + D[i+3]$

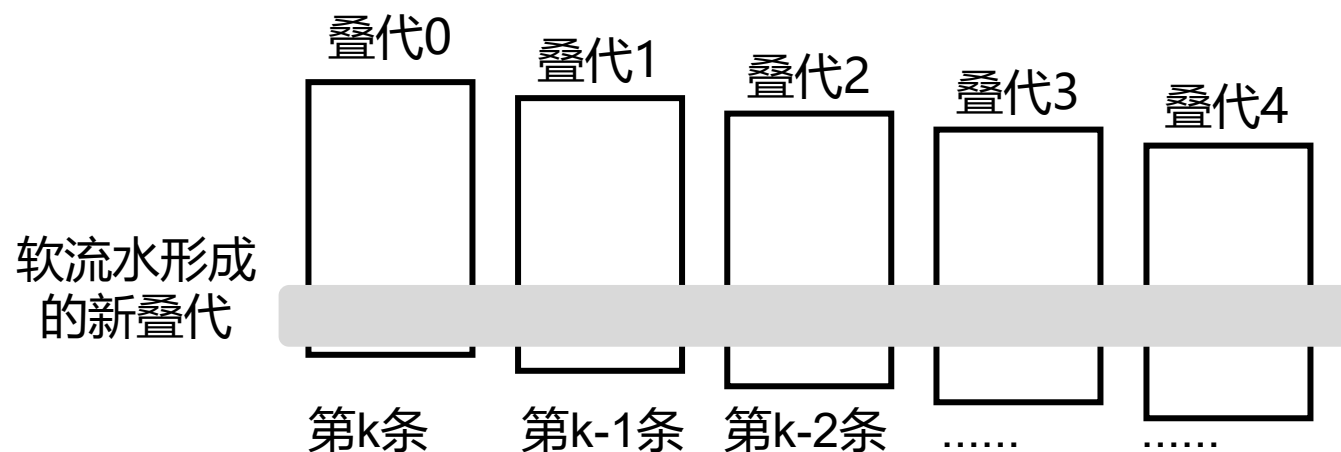


$B[i+1] = C[i] + D[i]$	$B[i+2] = C[i+1] + D[i+1]$	$B[i+3] = C[i+2] + D[i+2]$	$B[i+4] = C[i+3] + D[i+3]$
$A[i+1] = A[i+1] + B[i+1]$	$A[i+2] = A[i+2] + B[i+2]$	$A[i+3] = A[i+3] + B[i+3]$	$A[i+4] = A[i+4] + B[i+4]$

方法：从循环不同的叠代中抽取一部分指令拼成一个新的循环叠代

1. 简介

- 软流水技术的核心思想是从循环不同的叠代中抽取一部分指令（循环控制指令除外）拼成一个新的循环叠代。
- 目的
 - 将同一叠代中的相关指令分布到不同的叠代中
 - 将不同叠代中的相关指令封装到同一叠代中



软流水=虚拟的硬件流水线

注意：与硬件流水线不同的是，充满或排空的指令无法被封装到任何一个新的迭代中，只能放在新循环之前或之后。

6.5.2 软流水

2. 实例分析

例6.11 试用软流水技术处理例6.1中的循环，假设数组x有n个元素。

解 软流水需要从原循环的多个叠代中选择指令拼成新的循环，因此我们首先将原循环展开。

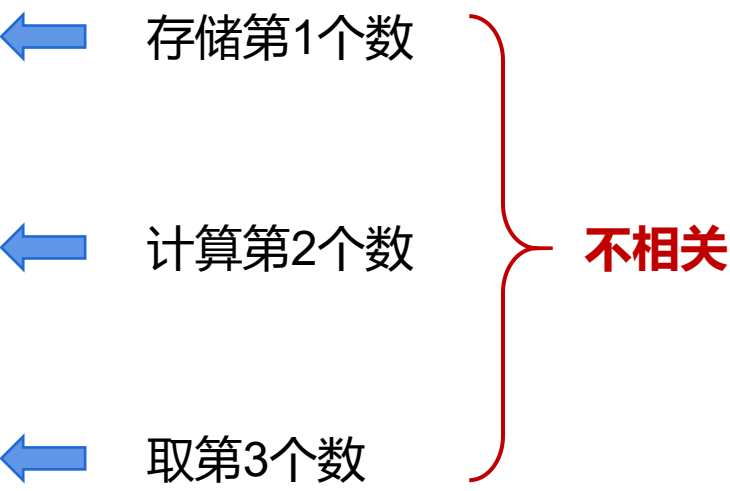
需要提前做:

- 1、取第1、2个数;
- 2、计算第1个数

需要补充做:

- 1、计算最后一个数
- 2、存储第n-1、n个数

叠代i:	; 修改x[i]并保存 L.D F0, 0 (R1) ADD.D F4, F0, F2 S.D F4, 0 (R1)
叠代i+1:	; 修改x[i-1]并保存 L.D F0, 0 (R1) ADD.D F4, F0, F2 S.D F4, 0 (R1)
叠代i+2:	; 修改x[i-2]并保存 L.D F0, 0 (R1) ADD.D F4, F0, F2 S.D F4, 0 (R1)



6.5.2 软流水

第 93 页

从这3个叠代中分别选出一条指令，与原有的循环控制指令拼在一起得到一个新的叠代。

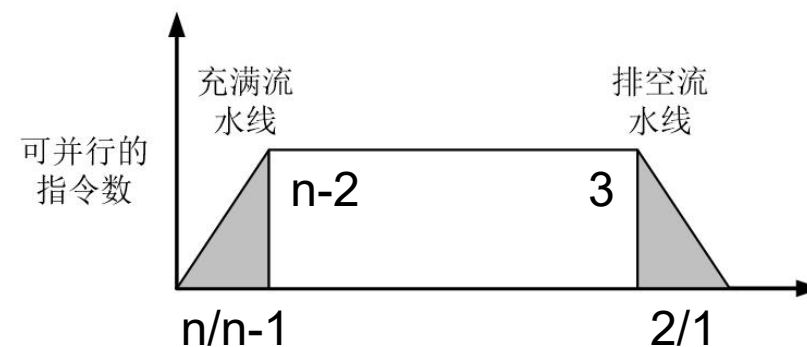
DADDUI	R1, R1, #-16	// I1: R1保存x[n-2]的地址
L.D	F0, 16 (R1)	// I2: 取x[n]
ADD.D	F4, F0, F2	// I3: $x[n] = x[n] + F2$
L.D	F0, 8 (R1)	// I4: 取x[n-1]
Loop: S.D	F4, 16 (R1)	// I5: 存x[i+2]
ADD.D	F4, F0, F2	// I6: $x[i+1] = x[i+1] + F2$
L.D	F0, 0 (R1)	// I7: 取x[i]
BNE	R1, R2, Loop	// I8
DADDUI	R1, R1, #-8	// I9: 填充分支延迟槽
S.D	F0, 8 (R1)	// I10: 存x[2]
ADD.D	F4, F0, F2	// I11: $x[1] = x[1] + F2$
S.D	F4, 0 (R1)	// I12: 存x[1]

6.5.2 软流水

第 94 页

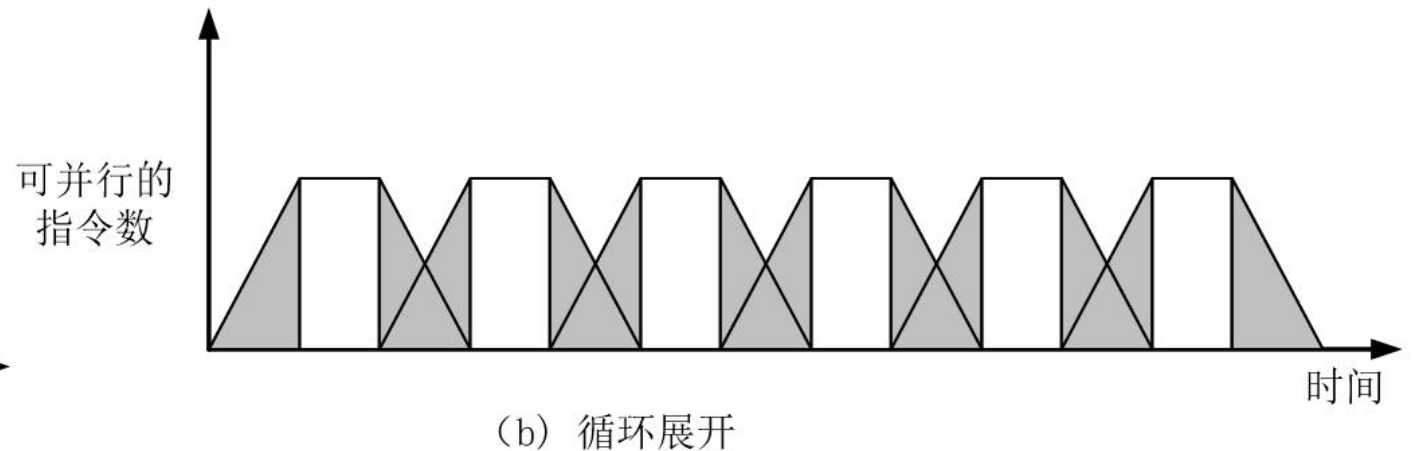
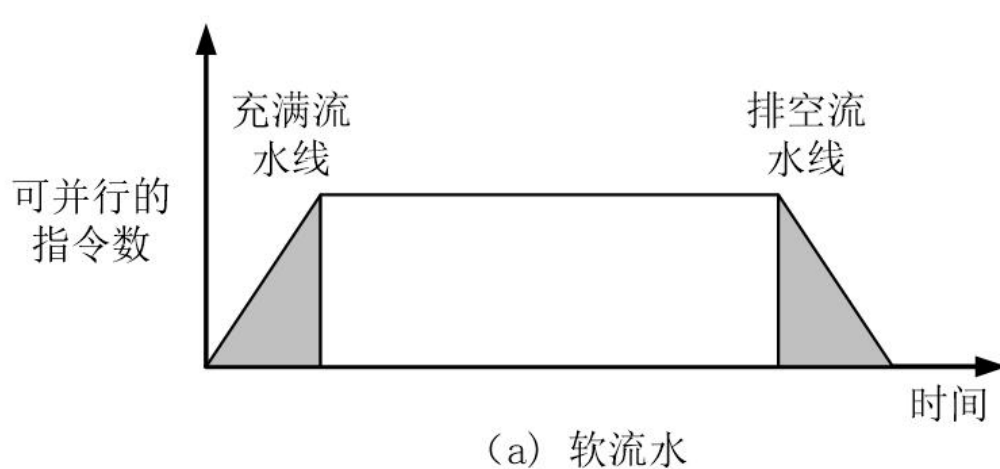
分析：

- 新循环的结束条件仍为： $R1 = R2$
- 新循环从元素 $x[n-2]$ 开始从头处理的，元素 $x[n]$ 与 $x[n-1]$ 只是从半途开始处理。
- 新循环的最后一次叠代只处理完元素 $x[3]$ ， $x[2]$ 刚被修改完结果尚未写回， $x[1]$ 刚被取出，需要被修改并写回。
- 新循环的每个叠代相当于流水执行了原循环的3个叠代。元素 $x[n/n-1/2/1]$ 的处理相当于充满和排空这条“流水线”。



3. 性能比较：软流水 vs. 循环展开

- 循环展开主要减少由分支指令和修改循环索引变量的指令所引起的循环控制开销。
- 软流水使叠代内的指令级并行达到最大。



1. 基本指令调度及循环展开：指令调度、循环展开及注意事项
2. 跨越基本块的静态指令调度：全局指令调度、踪迹调度、踪迹选择、踪迹压缩、超块调度、尾复制技术
3. 静态多指令流出：VLIW技术：VLIW vs. 超标量、多流出处理器 vs. 向量处理器
4. 显式并行指令计算EPIC：EPIC、非绑定分支、谓词执行、前瞻执行
5. 开发更多的指令级并行：循环携带相关、存储别名分析、软流水技术

1、试用软流水技术处理下面的循环。

```
L:      L.D      F0, 0(R1)           // F0 = 0(R1)
        ADD.D    F4, F0, F2          // F4 = F0 + F2
        S.D      F4, 0(R1)           // 将F4存入0(R1)
        DADDIU   R1, R1, #-4         // R1 = R1 - 4
        BNEZ     R1, L                // 若R1不为0, 转L
```

解：软流水处理后结果如下：

	DADDIU R1, R1, #-8	// R1保存x[n-2]的地址
	L.D F0, 8(R1)	// 取得x[n]
	ADD.D F4, F0, F2	// $x[n] = x[n] + F2$
	L.D F0, 4(R1)	// 取得x[n-1]
Loop:	S.D F4, 8(R1)	// 存x[i+2]
	ADD.D F4, F0, F2	// $x[i+1] = x[i+1] + F2$
	L.D F0, 0(R1)	// 取得x[i]
	DADDIU R1, R1, #-4	// 填充分支延迟槽
	BNE R1, 0, Loop	
	S.D F0, 4(R1)	// 存x[2]
	ADD.D F4, F0, F2	// $x[1] = x[1] + F2$
	S.D F4, 0(R1)	// 存x[1]

2、为了使用GCD测试方法判断一个循环是否含有存储别名，必须首先将循环索引变量的步长变为1。请先将下面的循环代码的步长变为1，然后用GCD测试方法判断其中是否存在存储别名。

```
for(i=2; i<=100; i+=2)
    a[i] = a[50*i+1];
```

解：修改结果如下：

```
for(i=1; i<=50; i++)
    a[2*i] = a[100*i + 1];
```

用GCD测试法， $a=2$ ， $b=0$ ， $c=100$ ， $d=1$ ， $GCD(c, a) = 2$ ， $(d-b) = 1$ 。由于2不能整除1，所以不可能存在存储器别名。

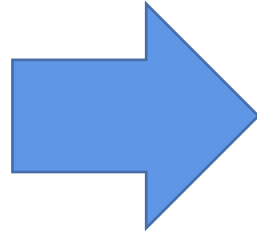
i=1		i=2		i=3		i=4		i=5	
a[2]	a[101]	a[4]	a[201]	a[6]	a[301]	a[8]	a[401]	a[10]	a[501]

应用题

第 99 页

3、试将基本块B1的第二条语句调度至基本块B2内，为了保证结果正确，可以加入一定的补偿代码。

B1: $x = x + 1$
 $y = x - y$
 if $x < 5$ goto B3
B2: $z = x * z$
 $y = y + 1$
 goto B5
B3: $y = 2 * y$
 $x = x - 2$



解: B1: $x = x + 1$
 if $x < 5$ goto B3
 B2: $y = x - y$
 $z = x * z$
 $y = y + 1$
 goto B5
 B3: $y = x - y$
 $y = 2 * y$
 $x = x - 2$

4、假设浮点流水线的延迟如下表所示。

产生结果的指令	使用结果的指令	延迟（时钟周期数）
浮点计算	另一个浮点计算	3
浮点计算	浮点store（S.D）	2
浮点load（L.D）	浮点计算	1
浮点load（L.D）	浮点store（S.D）	0

假设某VLIW处理器每个时钟周期可以同时流出5个操作，包括两个访存操作、两个浮点操作以及一个整数或分支操作。将下述代码循环展开，并调度到该VLIW处理器上执行。循环展开次数不定，但至少要是能够保证消除所有流水线“空转”周期，同时不考虑分支延迟。

```
Loop:    L.D          F0, 0(R1)      // 取一个向量元素放入F0
         ADD.D       F4, F0, F2      // 加上在F2中的标量
         S.D         F4, 0(R1)      // 存结果
         DADDIU      R1, R1, #-8     // 将指针减8（每个数据占8个字节）
         BNE         R1, R2, Loop    // 若R1不等于R2，表示尚未结束，转Loop继续
```

应用题

第 101 页

4、假设浮点流水线的延迟如下表所示。

访存操作1	访存操作2	浮点操作1	浮点操作2	整数分支操作
L.D F0, 0(R1)	L.D F6, -8(R1)	nop	nop	nop
L.D F10, -16(R1)	L.D F14, -24(R1)	nop	nop	nop
L.D F18, -32(R1)	L.D F22, -40(R1)	ADD.D F4, F0, F2	ADD.D F8, F6, F2	nop
L.D F26, -48(R1)	nop	ADD.D F12, F10, F2	ADD.D F16, F14, F2	nop
nop	nop	ADD.D F20, F18, F2	ADD.D F24, F22, F2	nop
S.D 0(R1), F4	S.D -8(R1), F8	ADD.D F28, F26, F2	nop	nop
S.D -16(R1), F12	S.D -24(R1), F16	nop	nop	nop
S.D -32(R1), F20	S.D -40(R1), F24	nop	nop	DADDUI R1, R1, #-56
S.D 8(R1), F28	nop	nop	nop	BNE R1, R2, Loop

第7章 存储系统